

IMPERIAL

IMPERIAL COLLEGE LONDON

DEPARTMENT OF COMPUTING

Shrinking SPHINCS₊: SNARK Aggregation for Post-Quantum Blockchains

Supervisor:

Prof. William Knottenbelt

Author:

Nicolas Schleicher

Second Marker:

Dr. Jonathan

Passerat-Palmbach

MEng Joint Mathematics and Computer Science

June 10, 2026

Acknowledgements

I would like to thank my supervisor, Prof William Knottenbelt and second marker, Dr Jonathan Passerat-Palmbach, for their unwavering support during our dissertation meetings. I would like to extend the same thanks to Conduition and the wonderful people at the Ethereum Foundation who showed tremendous interest in my work. Thank you to Emile, who generously ran my benchmarks on the Hetzner AX42-U server.

My journey through Imperial has had many ups and one or two downs. I would like to thank my parents, who worked unbelievably hard so that I could receive the privilege of studying at Imperial. I would like to thank Ria for being my all-reliable partner through thick and thin. I would not be the person I am without their never-ending support.

Some of my fondest memories of Imperial have been with my friends, old and new. Thank you to all the lunches, lectures and evening card games together. I look forward to discovering the next chapter after Imperial.

Abstract

Recent advancements in quantum hardware, error correction, and resource estimates have brought forward the expected arrival of "Q-day", the day a quantum computer can break elliptic-curve public keys using Shor's algorithm. Blockchains such as Bitcoin and Ethereum are especially vulnerable because their architecture is entirely reliant on quantum-vulnerable digital signatures.

Hash-based digital signatures such as SPHINCS+ offer a convincing post-quantum solution: unlike lattice-based alternatives, they rely only on well-studied hash primitives already embedded in blockchain infrastructure. However, SPHINCS+ signatures are 8–16 Kilobytes (KiB) by default, roughly 120 times larger than the elliptic curve signatures blockchains currently store per block. Direct replacement is therefore infeasible, and signature aggregation is required. This thesis fills that gap by providing an end-to-end evaluation of two Succinct Non-interactive Arguments of Knowledge (SNARK) based aggregation systems: CairoVM for design-space exploration, and the Ethereum Foundation's leanVM for a constraint-optimal prototype. We exploit leanVM's field-native optimisation primitives, including chunk-based bit decomposition and JMP-range specialisation for Merkle tree traversal, to substantially reduce prover cycle count. Together, these systems establish a baseline for hash-based SNARK aggregation and let us quantify the gap that separates it from production use.

CairoVM demonstrates that SNARK-based aggregation scales polylogarithmically, achieving 33% compression at 2,000 signatures. leanVM reaches compression break-even at 64 signatures and a $20\times$ compression ratio, at up to 140 signatures per second on cloud hardware. This break-even is better than that of LaBRADOR, the leading lattice-based aggregation scheme. A substantial throughput gap remains before practical deployment in Bitcoin or Ethereum; closing it will require SHA-256-compatible proving, recently standardised reduced SPHINCS+ parameter sets, and recursive proofs.

Contents

1	Introduction	8
1.1	Overview	8
1.2	Motivation	9
1.2.1	Why hash-based signatures?	9
1.2.2	Why SNARKs for signature aggregation?	10
1.2.3	Direct Cost of SPHINCS+ Signatures	11
1.3	Aims and Objectives	13
2	Background	15
2.1	Post Quantum Cryptography	16
2.2	Hash-based Post Quantum Signatures	17
2.2.1	The Winternitz one-time signature scheme	17
2.2.2	eXtended Merkle Signature Scheme (XMSS)	18
2.2.3	Forest of Random Subsets	20
2.2.4	SPHINCS+	21
2.2.5	SPHINCS+ parameters	22
2.3	Signature Aggregation	23
2.4	Hash-based SNARKs	24
2.4.1	SNARK Internals	26
2.4.2	STARKs vs hash-based SNARKs	26
2.4.3	Zero Knowledge Virtual Machines	27
2.4.4	Post-Quantum aggregation with SNARKs	27
2.4.5	LaBRADOR and Falcon Signature Aggregation	28
3	Methodology	29
3.1	Design Space Exploration in cairoVM	29
3.2	Constraint-optimal Proving with leanVM	31
4	Experiment: SPHINCS+ Design Trade-offs in CairoVM	33
4.1	Hypothesis	33
4.2	Implementation Details	34

4.2.1	SPHINCS+ Internal Hash Function	34
4.2.2	SPHINCS+ Aggregation Parameter Sets	35
4.3	SPHINCS+ Aggregation Scaling Results	37
4.3.1	Single Signature Circuit Results	37
4.3.2	SPHINCS+ Compression Results	40
4.4	Experiment Conclusion	40
5	Experiment: Constraint-Optimal SPHINCS+ with leanVM	43
5.1	Hypothesis	43
5.2	Implementation Details	44
5.2.1	SPHINCS+ Bit Decomposition	44
5.2.2	SPHINCS+ Tree Traversal	44
5.3	Results	47
5.3.1	Higher-Throughput Results on Cloud Hardware	48
5.4	Experiment Conclusion	50
6	Discussion	51
6.1	SNARK-based aggregation in Bitcoin	51
6.2	SNARK-based aggregation in Ethereum	52
6.3	Comparison with LaBRADOR Falcon Aggregation	52
7	Conclusion	53
7.1	Future Work	55
7.1.1	SPHINCS+ aggregation with SHA	55
7.1.2	NIST-approved SPHINCS+ parameter set	55
7.1.3	Recursive proof aggregation	55
7.1.4	Deployment within Blockchains	56
	Bibliography	62
8	Declarations	63
8.1	Use of Generative AI	63
8.2	Ethical Considerations	63
8.3	Sustainability	63
8.4	Availability of Source Code	64
A	Schnorr Signatures	65
A.1	Key Generation	65
A.2	Signing	65
A.3	Signature Verification	65
A.4	Schnorr Signature CairoVM Implementation	66

List of Figures

2.1	Hash chain from secret key sk_i to public key pub_i	18
2.2	A binary Merkle tree, where each parent node hashes the concatenation of its two children.	19
2.3	Merkle hypertree example where a leaf embeds a second-level Merkle tree.	20
2.4	High-level structure of a FORS hypertree used in SPHINCS+.	21
2.5	Interactive argument between a prover $\mathcal{P}$ and verifier $\mathcal{V}$. The prover alone holds the secret witness x ; each verifier challenge is answered with an oracle, and the verifier accepts only if the openings are consistent with C and y	25
3.1	Internal leanVM circuit: four AIR tables linked by a bus enforcing cross-table consistency.	31
4.1	WOTS signer grinding cost against target sum, trading signer work for fewer in-circuit hashes.	36
4.2	Cairo execution time for a single SPHINCS+ signature across hash functions and parameter sets.	38
4.3	Cairo execution cycles and resulting proof size for a single SPHINCS+ signature per variant.	39
4.4	Cairo prover time for verifying a single SPHINCS+ signature across hash-function variants.	39
4.5	Prover and execution time vs. number of aggregated SPHINCS+-Poseidon Mini signatures N	41
4.6	STARK proof size vs. number of aggregated SPHINCS+-Poseidon Mini signatures N , with polylogarithmic fit ($R^2 = 0.984$).	41
5.1	Optimised leanVM bit decomposition, replacing per-bit constraints with chunked range checks and a reduced sum.	45
5.2	JMP-range trick: an affine JMP into 2^n consecutive specialised functions, replacing n branches with one cycle.	47

5.3	SPHINCS+ Mini prover throughput against number of signatures aggregated using leanVM	48
5.4	leanVM aggregate proof size and compression ratio against the num- ber of aggregated SPHINCS+ Mini signatures.	49

List of Tables

2.1	SPHINCS+ parameter sets [1, 2].	23
2.2	Execution trace and transition constraints of an AIR encoding a simple Fibonacci VM, $r_{i,j}$ represents the i th register in the j th row. . . .	26
4.1	Native CPU hashing time (nanoseconds) for 512 bits of data across three hash functions [3].	34
4.2	Number of R1CS constraints for a circuit proving leaf knowledge in a Merkle tree of 2^{30} elements [4].	34
4.3	Cairo execution costs for SPHINCS+ signature verification with different hash functions.	35
4.4	Parameter sets for SPHINCS ⁺ -128s and two reduced instances [2]. . .	37
5.1	SPHINCS ⁺ -Poseidon aggregation: per-signature proof cost and compression ratio.	47

List of Algorithms

1	Merkle Tree Path Authentication	46
2	Merkle Tree Path Authentication Specialised with $i = 5$	46

Chapter 1

Introduction

1.1 Overview

Blockchain technology offers a digital infrastructure for money-based transactions secured with cryptographic schemes and economic incentives. This enables censorship-free, borderless transactions with open access without requiring a single central trusted party. If Cryptographically-relevant Quantum Computers (CRQCs) arise, the threat to the cryptographic schemes underpinning blockchain will be critical. This threat is possible due to Shor’s [5] algorithm, which enables polynomial-time discrete log computation on Elliptic Curve-based digital signatures. For this reason, the National Institute of Standards and Technology (NIST) have recently approved and standardised several post-quantum signature schemes, such as SPHINCS+ [6] and CRYSTALS-Dilithium [7] as replacements for traditional signature schemes. Industry is already acting on these standards: Google [8] and Cloudflare [9] are targeting full post-quantum resistance by 2029, underscoring the need for an early migration to beat Q-day.

However, these algorithms have several drawbacks; most post-quantum signature sizes are approximately two orders of magnitude larger than traditional non-quantum safe Elliptic Curve-based algorithms, and pose a major issue to blockchains looking to migrate to post-quantum solutions. Block space for transactions in blockchains is limited, and direct usage of post-quantum signatures would have severe effects on their scalability. One solution is signature aggregation, where multiple signatures are compressed into a single object attesting to the validity of multiple signatures. This would allow substantial savings and offset the scalability cost of post-quantum signatures in blockchain technology. This thesis explores the aggregation of the SPHINCS+ digital signature scheme using hash-based SNARKs, proof systems that allow a prover to arbitrarily prove the correct execution of any Turing-complete computation.

We focus specifically on the stateless transaction-signature setting, where signers are ordinary users and wallets that cannot be assumed to maintain reliable state; this is precisely the setting in which SPHINCS+, alone among standardised hash-based schemes, is applicable. The thesis provides a complete benchmark of SPHINCS+ signature aggregation and identifies the gap SNARK-based aggregation systems present for post-quantum blockchain scalability.

1.2 Motivation

The following section motivates the discussion along three lines. First, we motivate hash-based signatures such as SPHINCS+ in blockchains rather than other candidates. Secondly, we motivate the use of SNARK cryptography as a technique of signature aggregation and explain why hash-based signatures do not easily lend themselves to aggregation. Thirdly, we discuss different scenarios in blockchain with quantum-vulnerable cryptography and the cost of scalability if we directly replace the primitives with post-quantum signatures to demonstrate the importance of aggregation.

1.2.1 Why hash-based signatures?

As of April 2026, NIST has standardised two main digital signature schemes, SPHINCS+ and CRYSTALS-Dilithium. SPHINCS+ relies entirely on hashing functions for its security, while CRYSTALS-Dilithium is based on lattice cryptography. SPHINCS+ and its hash-based siblings rely on well-known hash functions such as SHA-2 and SHAKE [10], which have had decades of cryptographic scrutiny. Their relatively simple structure is easier to reason about than lattice-based cryptography. Additionally, blockchains already rely heavily on hash functions for security, so hash-based signatures are a natural alternative that do not add any extra cryptographic assumptions that may be broken in the future. Introducing lattice-based cryptography and isogenies such as SQISign [11] would add an extra assumption that could potentially be undermined. SQISign in particular, while compact at under 300 bytes, relies on the endomorphism ring problem, which has not been subjected to the same depth of cryptanalysis as hash functions and is not yet standardised at NIST Security Level 1. SPHINCS+ isn't the only hash-based scheme; in fact, SPHINCS+ is very closely related to XMSS [12], a much more compact hash-based signature with smaller signatures and faster signing times. However, XMSS is a stateful signature scheme and requires all signers to keep a counter of how many messages have been signed. If an XMSS signer loses track of this counter and signs a new message with a counter value it has already used, an external attacker can forge signatures. This poses a

critical problem if, for example, a user’s computer loses power and the true counter is lost. Statefulness can also introduce penalties to signing performance as one-time keys must constantly be synchronised between volatile and non-volatile memory [13]. This makes XMSS unsuitable for general-purpose transaction signing and user wallets, where signers cannot be assumed to maintain a reliable state.

Given that NIST has standardised SPHINCS+ [6] and that it is the only well-known stateless hash-based signature, it is a strong candidate for post-quantum signatures in blockchains.

1.2.2 Why SNARKs for signature aggregation?

As discussed earlier, SPHINCS+ offers a strong post-quantum security solution for Bitcoin and Ethereum, but due to its large 8–16 KiB signature sizes, it cannot be used as a direct replacement for elliptic curve signatures, which motivates the desire for signature aggregation. However, SPHINCS+ has no known algebraic properties that can be used to perform native aggregation. Hashing functions produce statistically random output, and so combining signatures with the same hashing function will break any relation which the verifier relies upon to verify the signature. This differs drastically from traditional signatures such as the Boneh, Lynn, Shacham signature (BLS) [14] whose aggregation techniques involve elliptic curve operations, which still preserve the core verification relationship. For that reason, we must turn to techniques external to SPHINCS+’s properties that can aggregate any computational relation.

SNARKs are cryptographic protocols which allow a prover to convince a verifier that they know a solution to a computational relation. The succinctness of SNARKs ensures that the verification time is less than that of checking the solution directly. This is a powerful tool because a prover can prove to a verifier the statement "I have verified that X signatures correctly link X (public key, message) pairs". The final proof is succinct with respect to the full concatenation of the X signatures.

For SPHINCS+ aggregation, we therefore require two properties from a SNARK: succinctness, so that the aggregated proof is smaller than the signatures it replaces, and post-quantum security, so that aggregation does not reintroduce the quantum vulnerability that SPHINCS+ was chosen to avoid. SNARKs encompass a wide range of cryptographic protocols to achieve succinctness, but not all of them meet the second requirement. Some SNARKs use elliptic-curve operations, which are vulnerable to quantum computers. For that reason, any SNARK used for SPHINCS+ aggregation should be hash-based, which ensures that the signature itself and its aggregation method rely only on hashing as a security assumption.

1.2.3 Direct Cost of SPHINCS+ Signatures

Having established SPHINCS+ as the post-quantum signature of choice and hash-based SNARKs as the aggregation mechanism, we now turn to why aggregation is needed at all. This section explores why Bitcoin and Ethereum cannot substitute SPHINCS+ directly for elliptic curve cryptography, either due to large signature sizes or incompatible native aggregation.

Bitcoin and Ethereum transaction signatures

Ethereum uses elliptic curve cryptography, specifically over the secp256k1 curve, to recover signers from a transaction signature, which is exactly 65 bytes long [15]. Bitcoin transactions, similarly, mainly use the secp256k1 curve; however, it includes the public key for an extra 64 bytes.

The SLH-DSA specification for SPHINCS+ is defined by NIST with a signature size of 7856 bytes for 128 bits of security [6]. This is approximately 120 times larger than the elliptic curve signatures that Bitcoin and Ethereum use. Smaller parameter sets for SPHINCS+, which only support a maximum of 2^{30} signatures before security degrades, have smaller signature sizes of approximately 4000 bytes [2].

Most blockchains store thousands of signatures per block. A block-size inflation of this magnitude will harm nodes and miners who ensure data availability and liveness of their blockchain. In Bitcoin, this would exclude an estimated 50%-80% of nodes, with more centralised networks and a higher barrier to entry [16].

Ethereum BLS Signature Aggregation

Ethereum’s consensus layer uses BLS signatures to maintain validator accountability and enables succinct verification of consensus signatures while storing a running list of exactly which validators have signed a given block. This allows the network to easily reward or punish validators while BLS aggregation enables the consensus network to scale [17].

BLS signatures use an elliptic curve pairing $e : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ whose bilinear property allows the aggregated signature to be reduced to a single equality check; however, they are inherently quantum-vulnerable. Given a bitfield of validators where entries are set to 1 if they have signed a given block and 0 otherwise, any node can calculate the aggregated public key $pk_G = \sum pk_i$. The aggregated signature is valid if and only if the following equality holds

$$e \left(\underbrace{\sum s_i}_{\text{Aggregate Signature}}, \underbrace{g}_{\text{Public Generator}} \right) \stackrel{?}{=} e \left(\underbrace{H(m)}_{\text{Hashed Message}}, \underbrace{\sum pk_i}_{\text{Aggregate Public Key}} \right)$$

Hash-based signatures, and other post-quantum resistant signature schemes for that matter, cannot be easily dropped in as replacements. No post-quantum scheme benefits from any such bilinear property which enables efficient BLS signature aggregation. By design, consensus signature verification and signing must be feasible well within Ethereum’s 12 second block time, leaving only a fraction of the slot, on the order of a second, for aggregation (Section 1.3). Paired with compact signature sizes, currently, no post-quantum signature scheme can provide a ready replacement. We note, however, that the consensus slot naturally supplies the per-signer state that stateful schemes require; consensus is therefore better served by XMSS than by SPHINCS+, a boundary we return to in Section 6.2. We include consensus here to delineate where stateless aggregation is and is not the right tool, and concentrate our contribution on the transaction-signature setting, where statelessness is mandatory.

Data Availability Commitments

To increase decentralisation, many blockchains support "light" clients, participants in the blockchain consensus who keep a small fraction of the blockchain and will rely on other clients to fetch data on demand. However, whenever a light client fetches data from another client, there is no guarantee that the data is correct. Ethereum uses the Kate, Zaverucha and Goldberg (KZG) commitment scheme [18] to commit to a "blob" of data. When a light client requests data, they can verify that the data is correct by checking the opening proof attached against the initial commitment. The opening proof is of constant length regardless of the size of the underlying data blob, making KZG a very powerful primitive for data availability. Unfortunately, the KZG setup requires generating a group element in $[\tau]\mathbb{G}$, where τ is known as "toxic waste". If an adversary manages to discover the value of τ , then they can forge arbitrary proofs. For that reason, the existence of a CRQC would render data availability commitments with KZG insecure. No polynomial commitment scheme with equivalent efficiency properties relying only on hash functions currently exists; the quantum-safety of data availability layers is therefore a separate open problem and is outside the scope of this thesis, which focuses on the signature layer.

1.3 Aims and Objectives

Having established SPHINCS+ as the post-quantum signature of choice and hash-based SNARKs as the aggregation mechanism, the remaining task is to measure how well this combination performs in practice. The primary objective of this thesis is therefore to establish a baseline for SPHINCS+ signature aggregation by benchmarking two concrete hash-based SNARK systems, cairoVM and leanVM, against SPHINCS+ and quantifying the gap that remains before practical deployment.

Bitcoin achieves an average transaction throughput of ≈ 3000 transactions [19] with median block confirmation times of ≈ 10 minutes [20]. Against this 10 minute window we set a deliberately conservative engineering target of aggregating thousands of signatures within a minute, a small fraction of the confirmation time that leaves ample headroom for propagation and mining; we treat this as a self-imposed objective rather than a hard protocol constraint. The final aggregated object should be on the order of the average size of a Bitcoin block of 2Mb [21]. Otherwise, a bloated aggregated object will have severe scaling problems on Bitcoin [16] and undermine its censorship-free and decentralised properties.

Ethereum has an average block time of 12 seconds [22], with an order of hundreds of transactions per block and an average block size of $\approx 150\text{Kb}$ [23]. Aggregation cannot consume the entire slot, which must also accommodate block proposal, distribution, signing, and propagation of the result. We therefore adopt a self-imposed target of aggregating within 1 second, leaving the remainder of the slot for the rest of consensus. This target is informed by the Ethereum Foundation’s own post-quantum consensus design [24], which allocates roughly one second of a four-second slot for aggregators to receive signatures. We note that their setting concerns validator attestations within consensus rather than transaction signatures; we extrapolate a comparable latency budget to the transaction-signing context as a deliberate engineering objective rather than a derived requirement.

Additional objectives include describing the challenges and optimisations that arise from SNARK-based aggregation of SPHINCS+, and a discussion on when SNARK-based aggregation of SPHINCS+ may be useful and situations where other post-quantum signatures are preferred.

While lattice-based aggregation schemes such as LaBRADOR [25] have been explored for Falcon and Dilithium signatures, no systematic benchmark of hash-only SNARK aggregation for SPHINCS+ exists. This thesis fills that gap, and in doing so identifies the conditions under which hash-based SNARK aggregation becomes practically viable.

The thesis comprises two stages of work. The first stage of work implemented SPHINCS+ aggregation with CairoVM [26] and explored several different implementations and variations of SPHINCS+ signatures. The second stage builds on the leanVM hash-based SNARK [27] for signature aggregation using optimal parameters and variations derived from the previous stage of work. Finally, we provide a baseline and tackle the issues and optimisations that are required for the practical deployment of hash-based SNARKs in blockchains such as Bitcoin and Ethereum.

To achieve this, the thesis proceeds as follows:

- We discuss current research in post-quantum signature aggregation to provide comparative context for this thesis (Chapter 2).
- We implement the original SPHINCS+ signature scheme using Cairo, a Rust-like language for proving computational statements with SNARKs (Chapter 4).
- We explore aggregation trade-offs with the SPHINCS+ signature scheme using different hash functions, parameter sets, and implementations (Sections 4.2.1–4.2.2).
- We contribute to an experimental optimised SPHINCS+ aggregation codebase, providing a state-of-the-art performance baseline for SPHINCS+ aggregation (Chapter 5).
- We justify, using empirical data, the areas in which SPHINCS+ aggregation is suitable for blockchain deployment (Chapter 6).

Chapter 2

Background

Blockchains are a decentralised network of computers which record a transaction ledger whose soundness heavily relies on security assumptions derived from low-level cryptographic primitives. More specifically, blockchains such as Bitcoin [28], and Ethereum [15] rely on two cornerstones of modern cryptography: digital signatures and hash functions. Digital signatures, such as the Elliptic Curve Digital Signature Algorithm (ECDSA) [29], enable users of blockchains to generate key pairs, consisting of public and private keys. Public keys are broadcast over the internet to link a user to an identity or address, while private keys are kept secret and used to sign transactions. This allows any third party, such as nodes in a blockchain network, to verify that a signed transaction belongs to a specified public key while preventing a malicious operator from forging signatures without access to the private key.

Hash functions such as the SHA [30] or BLAKE families are known as one-way functions: $H : \{0,1\}^* \rightarrow \{0,1\}^k$ that convert an arbitrary variable-length input into a fixed-length output. Hash functions have several useful properties, the most notable in blockchain systems is collision resistance and determinism. Determinism states that the same input always maps to the same output, and collision resistance ensures that it is sufficiently difficult to find two inputs x_1, x_2 such that $H(x_1) = H(x_2)$. Collision resistance implies pre-image resistance. Pre-image resistance states that it is computationally infeasible, given a hash output y to find an input x such that $H(x) = y$. Hash functions are used constantly for calculating blockchain addresses, transaction identifiers, state commitments and so on.

The security of digital signatures is derived from problems that are known to be difficult to solve in the context of classical computers. Although much of the literature focuses on the analysis of these problems, formal proofs for such problems do not exist, and this represents a fundamental vulnerability in modern cryptography. If these problems are found to be solvable within a reasonable time, the security of the cryptography scheme will collapse. For example, ECDSA relies on the Elliptic

Curve Discrete Logarithm Problem (ECDLP) [29], which states that given an elliptic curve with base point P and target point Q , it is computationally infeasible to find a scalar k such that $Q = [k]P$.

On the other hand, hash functions provide collision resistance, which has been subjected to decades of cryptanalysis with no public vulnerabilities found for algorithms such as SHA-2 and BLAKE2.

With the rise of quantum computers and quantum theory over several decades, Shor’s Algorithm [5] and Grover’s Algorithm [31] outline how Cryptographically Relevant Quantum Computers (CRQC) can be used to undermine or break cryptographic schemes that we use today. Shor’s algorithm specifically describes how a CRQC can reduce the security of elliptic-curve and integer-factorisation schemes to 0 bits of effective security [32]. Grover’s algorithm shows how CRQCs obtain a quadratic speed-up when brute-forcing pre-images of hash functions. For example, SHA-256, which classically offers 256 bits of pre-image security, requires approximately 2^{128} quantum gate operations to invert under a realistic fault-tolerant quantum model [33], thereby weakening the scheme without breaking it altogether.

CRQCs, therefore, pose asymmetric risk to digital signatures and hash functions. Digital signatures which use Elliptic Curve Cryptography are entirely broken and have severe consequences for blockchains, as CRQCs can reverse-engineer private keys from public keys. For example, 25% of Bitcoin addresses are vulnerable to this attack, holding an estimated worth of \$630 Billion USD [34]. Due to blockchain’s nature as an immutable ledger, a malicious actor can trivially perform forgery attacks on all vulnerable public keys, fundamentally undermining blockchain security. Another concern is an attack known as Harvest Now Decrypt Later (HNDL), where third parties can harvest and store encrypted data and public keys in anticipation of decrypting them when CRQCs become available.

2.1 Post Quantum Cryptography

Given the vulnerability of many digital signatures to quantum computers, the National Institute of Standards and Technology (NIST) issued a call for proposals for post-quantum cryptography in 2017. As of August 2024, two digital signature algorithms have been standardised by NIST: CRYSTALS-Dilithium (ML-DSA) [7] and SPHINCS+ (SLH-DSA) [6]. Each algorithm has different underlying assumptions, which are used in its security proofs.

CRYSTALS-Dilithium uses lattice-based cryptography, and its security is primarily derived from the Modular Short Integer Solution (MSIS) problem, which is outside the scope of this thesis; no classical or quantum algorithm is currently known to solve it efficiently. SPHINCS+ is a hash-based signature algorithm whose security

stems entirely from hash functions and their determinism, pre-image and collision resistance. SPHINCS+ suffers from large signature sizes of tens of kilobytes [6] compared to the tens of bytes that ECDSA signatures produce. Other notable competitors include SQISign [11], which relies on the endomorphism ring problem and achieves signatures under 300 bytes, among the smallest for post-quantum schemes.

SPHINCS+ is of particular interest to the blockchain community due to its sole reliance on hash functions for security, a primitive that blockchains already heavily rely on. Adopting SPHINCS+ reduces the risk of future breakthroughs severely weakening schemes such as Dilithium and SQISign, whose security assumptions are still immature compared to those of SPHINCS+. For that reason, SPHINCS+ has been chosen as the primary focus of this thesis.

2.2 Hash-based Post Quantum Signatures

SPHINCS+ builds on top of several hash-based schemes and algorithms, which are combined to produce a stateless, post-quantum secure digital signature algorithm. This section will provide a deeper look into SPHINCS+ and communicate the basic ideas that allow SPHINCS+ to function.

2.2.1 The Winternitz one-time signature scheme

The foundational building block of SPHINCS+ is the Winternitz One-time Signature (W-OTS) as described in [35]. W-OTS is a single-use hash-based scheme for signing arbitrary messages.

Key Generation. We choose the Winternitz Parameter $w \in \mathbb{N}$ alongside the number of keys and their lengths $n, L \in \mathbb{N}$. We use a random distribution to generate L n -bit secret keys.

$$(sk_0, \dots, sk_l) \leftarrow \{0, 1\}^{(n, L)} \quad (2.1)$$

We define a hash function and a recursive application

$$H : \{0, 1\}^* \rightarrow \{0, 1\}^n \quad (2.2)$$

$$f : \{0, 1\}^n \times \mathbb{N} \rightarrow \{0, 1\}^n \quad (2.3)$$

$$f(m, j) = H(f(m, j - 1)) \quad (2.4)$$

$$f(m, 0) = H(m) \quad (2.5)$$

Public keys are generated by repeated application of the hash function

$$pk_i = f(sk_i, w - 1) \quad (2.6)$$

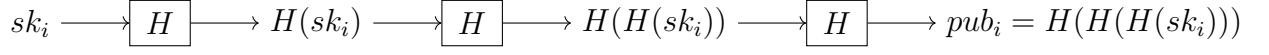


Figure 2.1: Hash chain from secret key sk_i to public key pub_i .

Signature Generation. Given a message $m : \{0, 1\}^n$:

1. split the message into L words of base- w , $a_i \in \{0, 1, \dots, w - 1\}$ for $i = 0, 1, \dots, L$
2. perform recursive hashing to obtain each signature element, $sig_i = f(sk_i, a_i)$ for all $i \in \{0, \dots, L - 1\}$

The full signature is then published as $(m, sig_1, sig_2, \dots, sig_L)$.

Signature Verification. Given a signature $(m, sig_1, sig_2, \dots, sig_L)$, we accept the signature if and only if

$$\forall i \in \{0, \dots, L - 1\} \quad pk_i = f(sig_i, w - 1 - a_i)$$

Note that this is a simplified W-OTS variant; the unmodified scheme is insecure, as an external party can create a valid signature for a message $m' : a'_i \geq a_i$ by calculating $f(sig_i, a'_i - a_i)$. This is mitigated by concatenating a checksum value $v = \sum w - 1 - a_i$ to the original message m to be signed alongside. With this, if an attacker tries to increase a value a_i , they must sign a smaller checksum, which requires finding a pre-image under the hash function H . This is assumed to be computationally infeasible. Other optimisations and tweaks are used in W-OTS and subsequent iteration W-OTS+, which is outside of the scope of this thesis.

W-OTS's defining property is single-use application; once a user signs more than one message with the same secret keys, they open themselves up to forgery attacks and allow any arbitrary attacker to sign messages on their behalf.

2.2.2 eXtended Merkle Signature Scheme (XMSS)

XMSS is an alternative many-time hash-based post-quantum signature scheme [12] to SPHINCS+. Although XMSS and SPHINCS+ share many similarities, they differ in that XMSS is a stateful signature scheme while SPHINCS+ is stateless. XMSS requires implementations to keep track of the number of signatures signed; any inconsistencies will leak information and allow attackers to forge signatures.

Deploying XMSS in blockchain systems is difficult because users must carefully maintain state through unforeseen events such as power loss or untracked backups. NIST recommends careful use of XMSS signatures and identifies several problems in their deployment for applications such as hardware wallets [36].

XMSS uses a key construct called Merkle trees, shown in Figure 2.2, which use hash functions to commit to an arbitrary amount of data with a fixed-length output. Given a vector of arbitrary data, we combine the data into a binary tree whose parents are the hash output of the concatenated children’s values. This produces a root value at the top, which can be published as a commitment. XMSS generates its public key in much of the same way, except each leaf value is a W-OTS public key. Therefore, during XMSS signing, the signature includes not only the W-OTS signature but also a Merkle tree membership proof showing that the W-OTS key belongs to the committed tree. Using Figure 2.2 as an example, the signer must provide ℓ_2 and H_2 , often known as an authentication path. The verifier then calculates H_1 and $H(H_1\|H_2)$ to verify that ℓ_1 belongs to the Merkle tree.

The security of the Merkle tree again relies on pre-image resistance of the hash function H . If the prover attempts to forge a proof of membership for an invalid value e , they must find an e' such that $H_1 = H(e\|e')$, which is computationally infeasible.

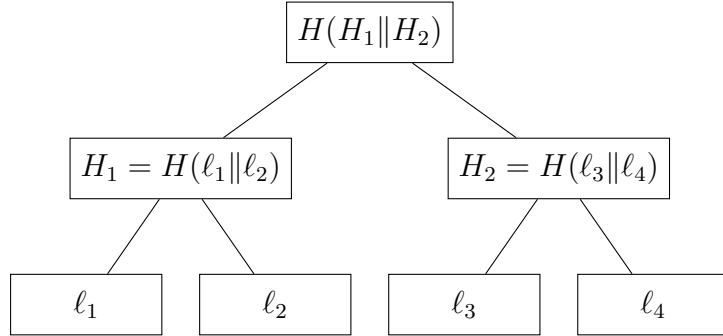


Figure 2.2: A binary Merkle tree, where each parent node hashes the concatenation of its two children.

A major drawback of XMSS is that if the desired capacity of signatures is too large, the Merkle tree must have a large height H , which has a knock-on effect on signature generation times. An alternative algorithm, XMSSMT [37], introduces hypertrees (Figure 2.3), built from multiple layers of binary trees. Lower layers are responsible for signing messages, while higher layers sign the roots of lower layers. This substantially increases the number of messages that XMSSMT can feasibly sign, as the signer does not need to provide an authentication path to the root of every sub-layer tree, only the trees that are used during signing.

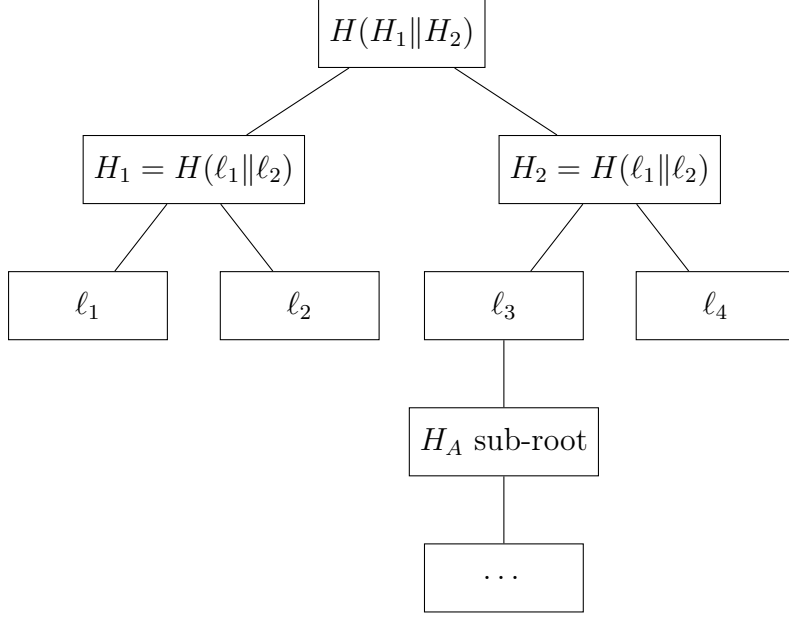


Figure 2.3: Merkle hypertree example where a leaf embeds a second-level Merkle tree.

2.2.3 Forest of Random Subsets

Unlike a one-time signature, Forest of Random Subsets (FORS) [6] provides a few-time signature scheme with a security boundary $C \in \mathbb{N}$ that provides a bound on how many messages can be signed using the same private/public key pair. If the number of messages signed exceeds C , the security of the signature scheme quickly degrades, hence why the scheme is known as a few-time scheme. FORS similarly uses Merkle hypertrees but stores private keys at leaf nodes instead of W-OTS public keys; these are revealed during signature generation.

Key Generation. The signer generates a list of k Merkle hypertrees, each of which contains 2^a leaf nodes. Each leaf node represents a private key, and the FORS public key is generated by hashing the concatenation of all k hypertree roots.

Signature Generation. Given a message M :

1. hash the message with a random value R to produce a digest $D = H(M||R)$
2. split D into k a -bit words $(d_1, \dots, d_k) \in \{0, 1\}^a$
3. for each d_j , take $l_{d_j}^j$, the d_j th index into the j th tree, and reveal it as the secret value along with the authentication path to the j th root value

A final signature is sent as $\{M, R, l_{d_1}^1, \sigma_1, \dots, l_{d_k}^k, \sigma_k\}$ where σ_i is the authentication path for the i th Merkle tree.

Signature Verification. Given a signature $\{M, R, l_{d_1}^1, \sigma_1, \dots, l_{d_k}^k, \sigma_k\}$:

1. split the digest into $(d_1, \dots, d_k)$

2. recalculate each Merkle root value using the opened values and authentication paths
3. hash the Merkle root values together and accept if and only if the output is the same as the FORS public key

The intuition behind the security of FORS comes from the Birthday Paradox [38]. For a single tree of size $N = 2^a$, if a user signs C messages, at most C secret keys will be revealed, which means the probability of re-using a secret key becomes $\approx \frac{S^2}{2N}$. If this is sufficiently small, it is computationally infeasible for an attacker to find a randomness value R' with a message M' so that the same secret keys are reused.

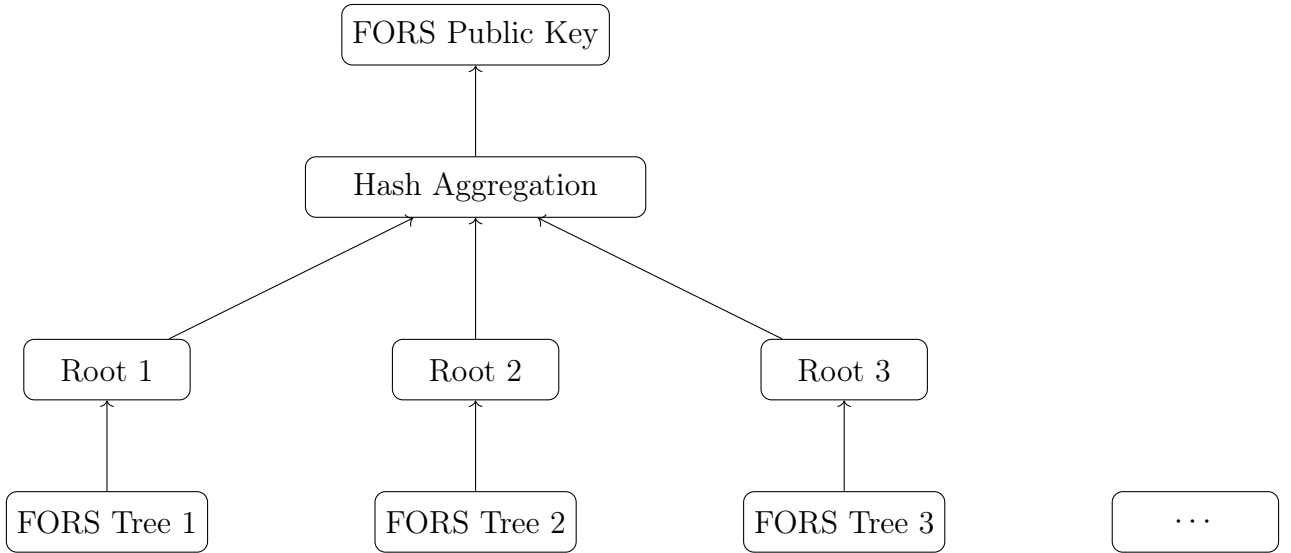


Figure 2.4: High-level structure of a FORS hypertree used in SPHINCS+.

2.2.4 SPHINCS+

SPHINCS+ ties together the ideas presented in Sections 2.2.1, 2.2.2 and 2.2.3 by combining these structures into a multi-layer hierarchy. SPHINCS+ resembles XMSS, with top-layer trees signing lower-layer root nodes using W-OTS keys. However, the main difference is that the last layer uses FORS rather than W-OTS to sign the message digest. As FORS public keys can be reused, this removes the final hurdle to statelessness and allows the use of a hash-based, many-time, stateless post-quantum signature scheme. The parameter set for SPHINCS+ under NIST Security Level 1 (128-bit security) is a public key size of 32 bytes, a private key of 64 bytes, and a final signature size of 8–16 KB [39].

2.2.5 SPHINCS+ parameters

The standard SPHINCS+ parameter sets described in Section 2.2.4 are designed to support up to 2^{64} signatures per key pair, a requirement originally mandated for all NIST post-quantum signature candidates. However, this upper bound is highly conservative for many real-world use cases. The original SPHINCS+ scheme targeted 2^{50} signatures, itself sufficient to issue one million signatures per second for thirty years under a single key [1]. The cost of supporting 2^{64} signatures is not free: the hypertree must be made substantially larger, directly inflating signature size and verification time.

Recognising this, Kölbl and Philipoom [1] conducted a broad search over the SPHINCS+ parameter space for sets supporting a smaller number of signatures while remaining otherwise compatible with the SLH-DSA specification. Their analysis shows that restricting the maximum signature count to $q = 2^{20}$ yields significant reductions in both signature size and verification overhead. Their proposed SPHINCS-128s-q20 parameter set achieves a signature size of just 3 264 bytes at NIST Security Level 1, a reduction of approximately 58% compared to the standardised SPHINCS+-128s, while maintaining 128-bit security [1]. NIST subsequently standardised a 2^{24} -signature SLH-DSA variant at Security Level 1 [40].

An important property of these reduced-signature parameter sets is that security degrades *gracefully* when the signature limit is exceeded. Unlike stateful schemes such as XMSS, where state reuse immediately leads to catastrophic key compromise [1], SPHINCS+ maintains meaningful security guarantees well beyond its nominal limit. With carefully chosen parameters, a key nominally rated for 2^{20} signatures retains over 100 bits of security even if 2^{30} signatures are issued [1]. This robustness to misuse is a material advantage over stateful alternatives in adversarial or high-availability environments.

Further reductions in signature size are possible through algorithmic optimisations to the SPHINCS+ construction itself. Kudinov and Nick [2] examine parameter selection and scheme variants tailored specifically to Bitcoin’s requirements, applying three recent improvements: SPHINCS+C, which compresses WOTS chain outputs; TL-WOTS-TW, a two-level Winternitz construction that reduces signing overhead; and PORS+FP, a replacement for FORS that avoids the index-collision costs inherent in the original few-time signing layer. Combining these optimisations with a reduced signature limit, they achieve signatures of approximately 4 036 bytes for a 2^{40} bound, 3 440 bytes for 2^{30} , and 3 128 bytes for 2^{20} , all at NIST Security Level 1 [2]. Notably, these optimised variants are between 16% and 18% smaller than vanilla size-optimised SPHINCS+ at equivalent signature limits [2].

Even with these optimisations, SPHINCS+-family signatures remain several kilo-

bytes in size, which represents a significant overhead relative to the 64-byte Schnorr signatures currently used in Bitcoin. This motivates the signature aggregation techniques discussed in Section 2.3.

Parameter Set	h	d	a	k	w	q	Sig. (bytes)
SLH-DSA-128s	63	7	12	14	16	2^{64}	7 856
SLH-DSA-128-24	22	1	24	6	4	2^{24}	3 856
SPHINCS-128s-q20	18	1	24	6	16	2^{20}	3 264
SPHINCS+C (W+C)	33	3	15	9	16	2^{30}	4 412

Table 2.1: SPHINCS+ parameter sets [1, 2].

2.3 Signature Aggregation

Despite their security, post-quantum signatures face a significant problem in blockchains. Most post-quantum signatures are several kilobytes large, and their use would substantially inflate congestion and competition for blockspace. For example, Bitcoin has an average block size of between 1–2 MB [21] using traditional elliptic-curve-based algorithms. With CRYSTALS-Dilithium, for blocks to remain within the 1 MB limit, the number of signatures would need to fall from ≈ 4000 to < 400 [41], substantially reducing transaction throughput.

A natural response to the large signature sizes of post-quantum signatures is signature aggregation, a technique by which signatures are compressed into a single object that attests to the validity of all constituent signatures without requiring a verifier to check each one individually. An aggregated signature that is substantially smaller than the concatenation of the signatures it covers reduces block space consumption, enabling both hash-based security and a compact on-chain footprint.

Succinct Non-interactive Arguments of Knowledge (SNARKs) are well-suited to signature aggregation. SNARKs allow a prover to prove an NP-relation to a verifier in poly-logarithmic time relative to the cost of directly checking that relation, enabling a prover to generate a proof attesting to the validity of N post-quantum signatures whose size is a fraction of the concatenated signatures. While many different types of SNARKs exist, hash-based SNARKs offer a post-quantum solution due to their construction from hash-based schemes. The following sections will provide a high-level overview of hash-based SNARKs, their properties, how they are constructed and their application to post-quantum signature aggregation.

2.4 Hash-based SNARKs

Hash-based SNARKs are a family of proof systems whose security derives entirely from collision-resistant hash functions. They allow a verifier to check a proof of a computational statement in time that is exponentially smaller than directly evaluating that statement. Additionally, zk-SNARKs are zero-knowledge variants of SNARKs that do not leak any information to the verifier other than the fact that a computational statement has been performed correctly. Many zero-knowledge proof systems are built as interactive protocols in which the prover and verifier exchange messages to establish the correctness of a computation. The Fiat-Shamir transformation [42] renders these protocols non-interactive: the prover generates a self-contained proof that any party can verify independently (hence the “non-interactive” in SNARKs).

Formally, SNARKs generate a proof that verifies an arithmetic circuit C evaluates to an output y in poly-logarithmic time with respect to the computation of the arithmetic circuit [43]. Arithmetic circuits are directed graphs where nodes represent arithmetic gates, such as addition/multiplication, and arrows represent output/input values between gates. Crucially, SNARKs provide *soundness* and *completeness* with overwhelmingly high probability. Completeness ensures that a proof of a true statement, e.g. that the 10th Fibonacci number is 55, will always be accepted by a verifier. Soundness ensures that a verifier will never accept a proof of a false statement. In practice, these properties hold with overwhelming probability, and the residual failure probability is negligible. SNARKs are therefore formally defined as *arguments* rather than proofs, as their soundness holds only computationally; throughout this thesis, we use the term “proof” for brevity.

Figure 2.5 illustrates the interactive protocol underlying these proof systems. The prover $\mathcal{P}$ holds the circuit C , the public input y , and a secret witness x satisfying $C(x) = y$, whereas the verifier $\mathcal{V}$ knows only C and y . Over several rounds, the verifier issues challenges and the prover replies with *oracles*: commitments and openings that the verifier can interrogate without learning x . The Fiat-Shamir transformation later replaces the verifier's random challenges with hash evaluations, collapsing this exchange into a single non-interactive proof.

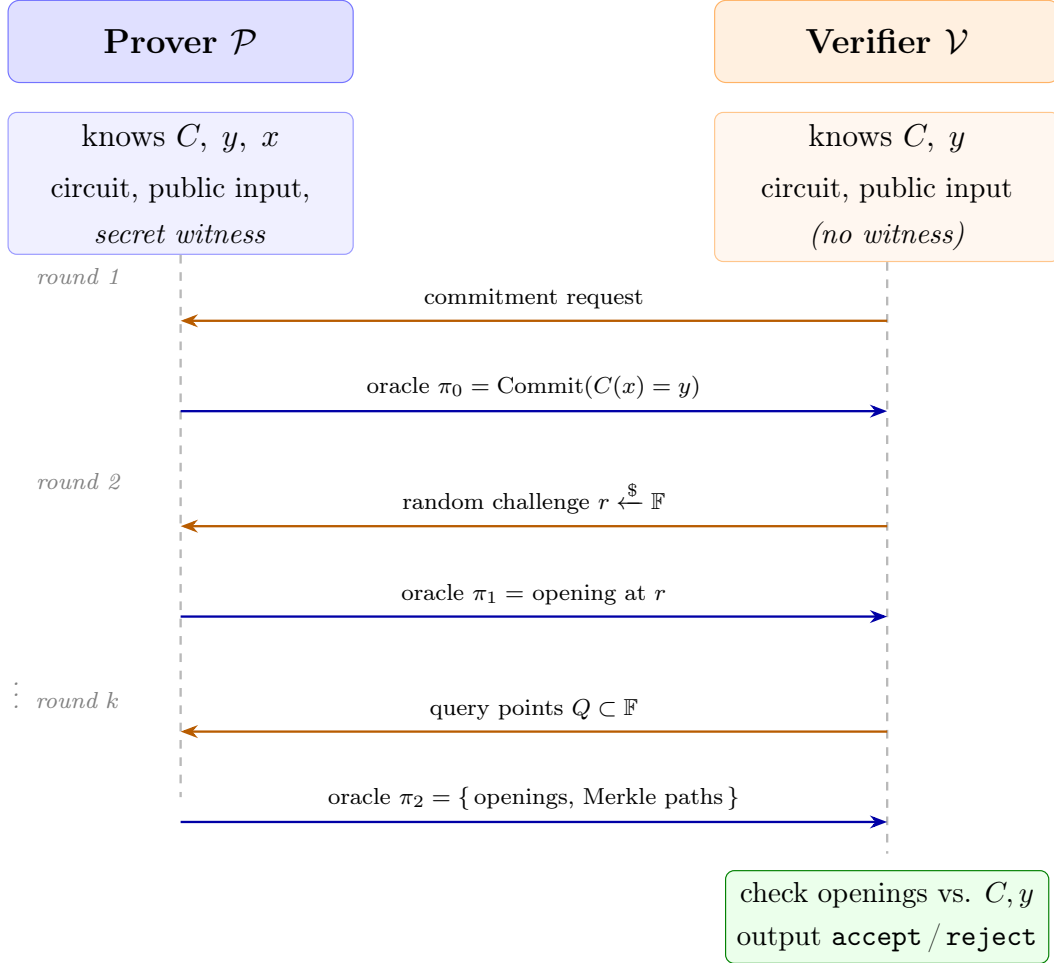


Figure 2.5: Interactive argument between a prover $\mathcal{P}$ and verifier $\mathcal{V}$. The prover alone holds the secret witness x ; each verifier challenge is answered with an oracle, and the verifier accepts only if the openings are consistent with C and y .

2.4.1 SNARK Internals

Once an arithmetic circuit has been chosen, the prover arithmetises the circuit into the Algebraic Intermediate Representation (AIR) [38]. The AIR is represented as a set of polynomials which represent the computation and the set of constraints that the computation must satisfy. These are combined to produce a polynomial which evaluates to 0 at certain points if and only if the computation correctly satisfies the constraints. Table 2.2 demonstrates an AIR arithmetisation of a Fibonacci calculation. AIR is one arithmetisation approach; alternatives include Plonk-ish systems and Quadratic Arithmetic Programs [44, 45].

t	pc_t	r_0	r_1	r_2	C_{pc}	C_{r_0}	C_{r_1}	C_{r_2}
0	0	2	1	1	$pc_1 - (pc_0 + 1)$	$r_{0,0} - (r_{1,0} + r_{2,0})$	$r_{1,1} - r_{2,0}$	$r_{2,1} - r_{0,0}$
1	1	3	1	2	$pc_2 - (pc_1 + 1)$	$r_{0,1} - (r_{1,1} + r_{2,1})$	$r_{1,2} - r_{2,1}$	$r_{2,2} - r_{0,1}$
2	2	5	2	3	$pc_3 - (pc_2 + 1)$	$r_{0,2} - (r_{1,2} + r_{2,2})$	$r_{1,3} - r_{2,2}$	$r_{2,3} - r_{0,2}$
3	3	8	3	5	$pc_4 - (pc_3 + 1)$	$r_{0,3} - (r_{1,3} + r_{2,3})$	$r_{1,4} - r_{2,3}$	$r_{2,4} - r_{0,3}$
4	4	13	5	8	$pc_5 - (pc_4 + 1)$	$r_{0,4} - (r_{1,4} + r_{2,4})$	$r_{1,5} - r_{2,4}$	$r_{2,5} - r_{0,4}$

Table 2.2: Execution trace and transition constraints of an AIR encoding a simple Fibonacci VM, $r_{i,j}$ represents the i th register in the j th row.

The proving time for most SNARKs is quasilinear in the number of rows that a program produces [43], so the prover cost is approximately proportional to the size of the computation. This places additional constraints on programs that are not naturally compatible with arithmetisation: operations that are cheap on modern CPUs (bit shifting, masking, floating-point arithmetic) become substantially more expensive in SNARKs. Some of these operations are reduced through techniques such as lookup tables for range checking [46]. Hashing algorithms, such as SHA-256, are optimised for bit-based computation but perform poorly in SNARK arithmetic circuits. For this reason, zero-knowledge programs prefer arithmetic-friendly hash functions such as Poseidon [4] or Rescue-Prime [47], which are cheaper to prove.

2.4.2 STARKs vs hash-based SNARKs

Ben-Sasson et al. [48] introduced the idea of Scalable Transparent Arguments of Knowledge (STARKs), an early example of hash-based SNARKs. Within this thesis, we will use the terms STARKs and hash-based SNARKs interchangeably. In many STARK-based systems, once a program has been arithmetised into AIR, the prover commits to the trace polynomial using Merkle trees and opens values at certain points with attached Merkle tree proofs. The prover also performs a Fast Reed-Solomon (FRI) proximity test [49], which proves to the verifier that the committed

polynomial has low degree, preventing a dishonest prover from inflating the number of satisfying points and thereby producing an unsound proof. In fact, CairoVM uses STARKs as the underlying proof system.

2.4.3 Zero Knowledge Virtual Machines

A significant drawback of writing provable programs for SNARKs is that developers must be able to generate the circuit by hand. The correctness and soundness of circuits must be manually audited, which incurs increased labour costs [26]. Another approach is to implement a zero-knowledge VM (zkVM), which provides a layer of abstraction between developers and the proof system used. Programs can be written in any programming language and compiled into an assembly program that uses only the Instruction Set Architecture (ISA) supported by the zkVM; the zkVM then produces a proof asserting that “if program A is executed in the zkVM, the output is y ”, rather than the lower-level statement “circuit C representing program A produces output y ”. From now on, we will refer to the program A as the guest program. zkVMs may feature custom ISAs, such as CairoVM [26], or support standard architectures like RISC-V [50]. Supporting traditional ISAs means that the zkVM can prove the correct execution of any program which can be compiled into that ISA; however, this comes with added complexity and longer proving times.

This thesis employs two zkVMs that represent different points on this trade-off: CairoVM, a custom-ISA zkVM from StarkWare used for design-space exploration of SPHINCS+ aggregation, and leanVM [27], a hash-native zkVM from the Ethereum Foundation optimised for efficient Poseidon operations, used for production-oriented benchmarking. Their design decisions and operational details are described in Chapter 3.

2.4.4 Post-Quantum aggregation with SNARKs

As noted in Section 2.3, large post-quantum signatures impose a substantial cost on blockchain scalability. The aggregate signature footprint of a block grows linearly with the number of transactions, whereas a SNARK proof grows only polylogarithmically [43]. Block builders and users can therefore collaborate to generate a single SNARK proof attesting to all signatures in a block, constraining the otherwise linear growth in on-chain data caused by post-quantum signatures.

Hash-based SNARKs have strong support for recursive SNARKs, i.e. a SNARK proof included in the verification of another SNARK, enabling recursive batching. This is useful for custodial exchanges that issue many transactions: an exchange can generate a SNARK proof over its own signatures, and combine it with other proofs to produce a SNARK attesting to a large number of transactions.

2.4.5 LaBRADOR and Falcon Signature Aggregation

Falcon is a lattice-based signature scheme from NIST’s standardisation Round 4, with security resting on the hardness of the Short Integer Solution problem over NTRU lattices. It operates over a power-of-two cyclotomic ring $\mathbb{Z}[X]/(X^d + 1)$ of degree $d \in \{512, 1024\}$, targeting 128-bit and 256-bit security respectively. Signing uses a trapdoor sampler to produce a pair of short polynomials satisfying a norm bound, and Falcon-512 achieves signatures of just 666 bytes, making it among the most compact post-quantum signature schemes available.

LaBRADOR [25] is a lattice-based non-interactive argument of knowledge published at CRYPTO 2023, whose security rests on the Module-SIS assumption. Similarly to SNARKs, LaBRADOR encodes a computation as a constraint system, here Rank-1 Constraints (R1CS) rather than AIR, and produces a succinct proof of satisfiability via a recursive amortisation strategy, rendered non-interactive by the Fiat-Shamir transformation [42]. Unlike STARKs, whose security derives from hash functions, LaBRADOR’s security is lattice-based, and it achieves proof sizes of around 58 KB for circuits with 2^{20} constraints at 128-bit security [25].

Årdal et al. [51] formalise how to aggregate Falcon signatures using LaBRADOR at CRYPTO 2024: the verification of N Falcon signatures is expressed as an R1CS relation, and a single LaBRADOR proof attests to the validity of all N signatures at once. Concretely, the aggregate signature becomes shorter than a naive concatenation of individual signatures for roughly 110 or more signatures [51].

Chapter 3

Methodology

3.1 Design Space Exploration in cairoVM

Post-quantum hash-based signatures are designed for classical hardware operating on bytes and non-algebraic hash functions such as SHA-256. This is fundamentally incompatible with field-based SNARK implementations, and implementing the standardised SLH-DSA specification of SPHINCS+ may prove untenable for SNARKs.

The benchmarks record results across three primary metrics: Cairo step count, prover time, and proof size. Rather than treating these as independent concerns, we observe that all three are coupled through the structure of its Stwo prover. Cairo steps determine the length of the STARK execution trace, which in turn drives prover time roughly linearly. In contrast, proof size and prover time both scale *logarithmically* in trace length due to FRI’s recursive commitment structure [48]; batching N signatures into a single trace thus amortises the fixed proof overhead across all N verifications. This logarithmic scaling is the central property that makes the batched STARK setting attractive for blockchain applications, and confirming and quantifying it empirically is a primary goal of the experiment.

We explore the parameter sets provided in [2] for SPHINCS+ and observe the signing and proving trade-offs across variants. Additionally, we explore SPHINCS+ variants using the SHA-2, Blake2s, and Poseidon hash functions.

After building on previous work, we found that the Poseidon algebraic function offers substantially superior prover throughput compared to other hash functions, which motivates its isolated use for the rest of the thesis. We adopt Poseidon as a pragmatic choice to establish a feasibility baseline and upper bound on aggregation performance; its suitability as a production cryptographic primitive (particularly for Bitcoin) is a separate question, addressed in the discussion.

We also reduced the effective number of Poseidon invocations by memoising intermediate sponge states. Poseidon is constructed as a sponge function [4], maintaining

an internal state that is updated by absorbing input elements and compressed to produce a hash output. In Cairo, the Poseidon hash function accepts field elements (`felt252` values) one at a time, so each SPHINCS+ hash call requires multiple Poseidon absorb steps: first the public key seed (`pk_seed`), then up to six domain-separation elements, followed by the message input. While those domain-separation elements are not globally constant across an entire verification, they remain constant across a large number of consecutive hash calls, for example, throughout all hashes within a single Merkle tree layer. By pre-absorbing `pk_seed` and the stable domain-separation prefix once per such group and retaining the resulting sponge state, subsequent hash calls within that group can begin from the cached state and absorb only the elements that vary, substantially reducing the total number of Poseidon operations required per verification.

Finally, an important observation is that in CairoVM, the main overhead arises from the number of Poseidon hash calls; reducing this number substantially increases prover throughput and decreases proof sizes. We can selectively choose parameters for SPHINCS+ that place a greater burden on the signer but significantly reduce the cost of signature verification. Off-circuit computation is orders of magnitude cheaper than on-circuit computation, so a modest increase in signer workload can yield large prover speedups.

3.2 Constraint-optimal Proving with leanVM

The Cairo programming language is fundamentally a smart contract language, designed to prove computations in a blockchain context, which is a different domain from hash-based signature aggregation. The leanVM project, spearheaded by the Ethereum Foundation, is a zkVM designed for highly efficient hash operations with SNARKs. This is achieved using special tables for Poseidon hash operations, which can be connected to the main execution trace for consistent lookups.

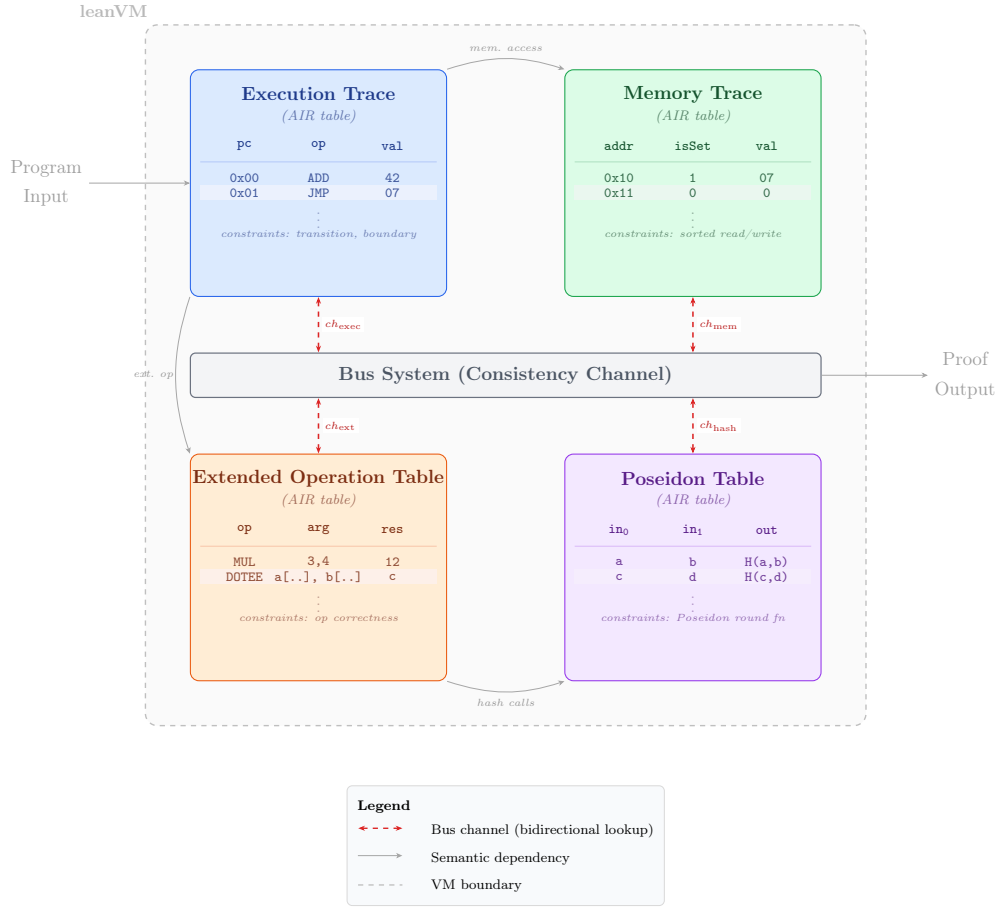


Figure 3.1: Internal leanVM circuit: four AIR tables linked by a bus enforcing cross-table consistency.

The leanVM system provides a low-level domain-specific language, which was used to implement SPHINCS+. The default SPHINCS+ specification requires many byte-specific operations, which are expensive to prove in leanVM. Building on the optimisations explored with CairoVM, we experiment with field-native techniques to avoid bit decomposition and use a JMP-range trick to substantially reduce prover cycle count.

The first field-native optimisation follows from the observation that bit decomposition has extremely high overhead. Bit decomposition must be properly constrained by verifying that each bit takes only the value 0 or 1 and that the final bit string corresponds to the original value. The key observation enabling a significant reduction of prover cost is that increasing the size of the program bytecode is much cheaper than performing dynamic branches repeatedly. Assume we have a function f with input $i \in \{0, \dots, 255\}$. Rather than dynamically calling $f(i)$, we can instead compile 256 separate functions f_i whose internal code does not depend on any runtime value. We lay each function f_i , contiguously in bytecode, and we perform a single **JMP** to the correct f_i depending on the value i . A **JMP** instruction only adds a single constraint cycle for the prover, but by forcing each f_i to not have any run-time inputs, we may save hundreds or thousands of cycles internally. This **JMP-range** trick allows us to entirely avoid bit decomposition.

In SNARK design, expensive computations can be offloaded outside the prover circuit when a cheap verification method confirms correctness. Whenever we are forced to perform bit decomposition, we offload the computation off-circuit and then ensure that the untrusted result is correct. This is much cheaper than computing bit decomposition within the circuit.

Additionally, as the 2^{64} signature limit far exceeds the requirements of most blockchain use cases, we implemented a 2^{30} signature limit, yielding a 66% reduction in Poseidon hash calls and a corresponding decrease in verification costs.

Chapter 4

Experiment: SPHINCS+ Design Trade-offs in CairoVM

4.1 Hypothesis

This experiment serves as the design-space exploration stage of the thesis, pursuing two goals. The first is to establish whether hash-based SNARK aggregation of SPHINCS+ is feasible at all, by demonstrating that aggregating thousands of signatures yields meaningful savings over naive concatenation within a deployment-realistic time frame. The second is to characterise the design-space trade-offs that govern this cost, so that the most favourable configuration can be carried forward to the constraint-optimal implementation in Chapter 5.

We hypothesise that the dominant cost of SPHINCS+ aggregation lies in the hash invocations performed inside the circuit, and that this cost can be reduced along two largely independent axes. The first axis is the choice of internal hash function: although SNARK-friendly algebraic hashes such as Poseidon are slower than SHA-2 or Blake2s on a native CPU, we expect them to be substantially cheaper once expressed as arithmetic constraints. The second axis is the SPHINCS+ parameter set: by lowering the signature limit and applying target-sum grinding, we expect to shift work from the prover to the signer and thereby reduce the number of in-circuit hashes per verification. Quantifying both axes against the Bitcoin and Ethereum targets of Section 1.3 identifies how far current proving systems fall short of practical deployment.

We deem the experiment a success if it aggregates signatures at volumes relevant to Bitcoin or Ethereum and demonstrates a clear compression improvement over naive concatenation.

All results presented below were generated on an AMD EPYC 7401P 24-Core Processor.

4.2 Implementation Details

4.2.1 SPHINCS+ Internal Hash Function

We evaluated three hash functions: SHA-2, Blake2s, and Poseidon. SLH-DSA is standardised with SHA-2, but Blake2s offers significant performance advantages on native CPU, while Poseidon is an algebraic hash function purpose-built for SNARKs.

Hash Function	Native CPU Time (ns)
SHA-256	319.1
Blake2s	213.3
Poseidon	19,944.0

Table 4.1: Native CPU hashing time (nanoseconds) for 512 bits of data across three hash functions [3].

As outlined in Table 4.1, Poseidon is substantially slower than its non-algebraic counterparts on a traditional CPU. However, in R1CS circuit representation, the performance ranking reverses, as described in the Poseidon paper [4] and shown in Table 4.2.

Hash Function	Arity	Width	R_F	R_P	Total Constraints
POSEIDON-128	2:1	3	8	57	7,290
	4:1	5	8	60	4,500
	8:1	9	8	63	4,050
SHA-256	510	171	—	—	826,020
Blake2s	510	171	—	—	630,180

Table 4.2: Number of R1CS constraints for a circuit proving leaf knowledge in a Merkle tree of 2^{30} elements [4].

The thesis builds upon previous work that implemented an early version of SPHINCS+ using both SHA-256 and Blake2s. Cairo offers a Poseidon pre-compile, a highly efficient implementation of the Poseidon hash function, which this thesis used. Unlike Blake2s and SHA-256, which have the same output value $B \in \{0, 1\}^{256}$, the Cairo Poseidon function outputs a field element $F \in \mathbb{F}_p$ where $p = 2^{251} + 17 \times 2^{192} + 1$. At 252 bits, this falls short of the 256 bits SPHINCS+ expects, requiring complete internal rewiring to ensure correct signature generation and verification.

Hash function performance is shown in Table 4.3. Both Poseidon and Blake2s variants are provable, but SHA-256 exceeds Cairo’s memory limit of 2^{27} addresses. Blake2s values are corrected from the original report; Poseidon results are from our implementation; SHA-256 steps and addresses are approximate [52].

Hash Function	Instance	#Verify Hashes	Cairo Steps	Memory Addresses	Proven
Blake2s	reduced	8,317	4,648,242	4,441,149	yes
SHA-256	full	2,088	195,000,000	134,000,000	no
Poseidon	full	2,088	153,630	182,593	yes
	reduced	713	62,023	75,782	yes

Table 4.3: Cairo execution costs for SPHINCS+ signature verification with different hash functions.

The key finding is that CairoVM’s lack of efficient SHA-256 support makes standard SLH-DSA SPHINCS+ aggregation infeasible; however, using other hash functions offers substantial gains, with modest cost with Blake2s. Poseidon, with a fraction of the Cairo steps, directly translates to the best prover performance.

The number of Cairo steps correlates directly with the number of hashes required for signature verification, suggesting that the physical hash function itself dominates SPHINCS+ aggregation cost; reducing hashes per verification is therefore a strong lever for improving performance.

4.2.2 SPHINCS+ Aggregation Parameter Sets

Another important observation within practical SPHINCS+ deployment is that the 2^{64} signature limit supported by SLH-DSA is much too large for most blockchain use cases. Most users will sign fewer than 2^{20} signatures in their lifetime, enabling substantial reductions in the signature limit with corresponding gains in signature size, verification cost, and proving times.

In this section, we examine the different parameter sets introduced in [2] and their trade-offs in SPHINCS+ aggregation.

As hypothesised earlier in Section 4.2.1, the main bottleneck in CairoVM aggregation of SPHINCS+ arises from the number of hash calls. However, SPHINCS+ can be modified to include signer-based grinding [2] to reduce the number of hash calls that must be aggregated by the prover. Each time a SPHINCS+ signer signs a message, they attach additional randomness which has been generated to satisfy "grinding" conditions. Specifically, this thesis explores target-sum grinding as an optimisation method.

Target-sum grinding attempts to find a random value so that the corresponding WOTS encoding vector sums to a target value. The guest program will regenerate the same WOTS encoding vector with the given randomness and ensure that the target value is reached. This can substantially reduce the prover aggregation work,

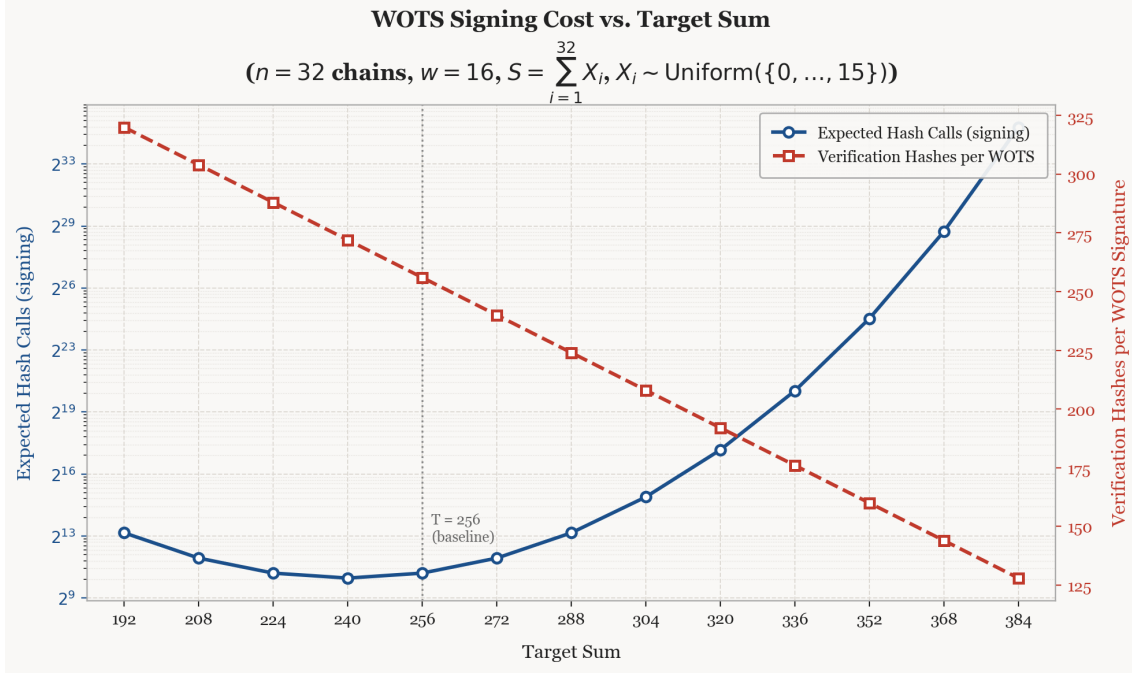


Figure 4.1: WOTS signer grinding cost against target sum, trading signer work for fewer in-circuit hashes.

as the guest program must perform

$$\sum_{i=0}^{chains} encoding_{max} - encoding[i]$$

hash calls per WOTS call. However, this comes at the cost of the signer, who has to perform an exponentially larger number of calls. The trade-offs between the prover and signer can be shown in Figure 4.1. Intuitively, we can lean on high WOTS target sums as it is exponentially cheaper to perform Poseidon hash calls outside of a zkVM than inside. To reduce the number of Poseidon hash calls, we opted to use the 304 targetsum variant from [2], which requires 713 Poseidon hashes for a single SPHINCS+ signature verification and approximately 2^{15} signer grinding attempts per WOTS call.

The alternative parameter sets also achieve smaller signature sizes and verification costs by scaling down on the size of the Merkle tree and FORS structures described in Section 2.2.3 and Section 2.2.2. The parameter sets are summarised in Table 4.4.

CairoVM Cycle Optimisations

A substantial reduction in Cairo cycles between our implementation and previous implementations also arises from field-native optimisations by migrating most integer-

Instance	h	d	a	k	w	$S_{w,n}$	Sig. Size (B)	# Verif. Hashes
SPHINCS ⁺ -128s (full)	63	7	12	14	16	–	7,872	2,088
SPHINCS ⁺ Mini (2^{30} , $T = 304$)	33	3	15	9	16	304	4,412	713
Reduced (2^{40} , $T = 2040$)	40	5	14	11	256	2040	4,036	10,380

Table 4.4: Parameter sets for SPHINCS⁺-128s and two reduced instances [2].

based operations into field-native ones. As CairoVM operates over a prime field, all `u32` or `u64` operations must have a range-check after each operation to ensure that the resulting value lies within 2^{32} or 2^{64} respectively [26]. This leads to a large increase in the number of constraints, most of which are not necessary, as most operations can be done directly within the native field.

A second optimisation targets the structure of SPHINCS+ hash calls in Cairo. Poseidon is constructed as a sponge function [4], maintaining an internal state that is updated by absorbing input elements and compressed to produce a hash output. The Cairo Poseidon hash function exposes this interface via `update` and `finalize` functions [26], accepting field elements (`felt252` values) one at a time, so each logical SPHINCS+ hash call requires multiple Poseidon absorb steps. Specifically, each call must first absorb the public key seed (`pk_seed`), then up to six domain-separation elements [39], before absorbing the message input. The naive approach rebuilds this full prefix on every call; however, while the domain-separation elements are not globally constant across an entire verification, typically three or four of them remain constant across a large number of consecutive hash calls, such as all hashes within a single Merkle tree layer. By pre-absorbing `pk_seed` and the stable domain-separation prefix once per such group and caching the resulting sponge state, each subsequent hash call within that group begins from the cached state and absorbs only the elements that vary. Since Cairo step count is the primary driver of prover cost, eliminating these redundant absorb steps across the thousands of hash calls in a single signature verification has a direct and significant effect on proving time and proof size.

4.3 SPHINCS+ Aggregation Scaling Results

4.3.1 Single Signature Circuit Results

We demonstrated the feasibility of SPHINCS+ aggregation using CairoVM after choosing the parameter sets defined in Section 4.2.1 and Section 4.2.2. We implemented a SPHINCS+ verification circuit in CairoVM, with initial comparisons against the SPHINCS-BTC Blake2s implementation from [52]. We compare the Cairo steps, proof size for a single SPHINCS+ signature, and prover throughput for each

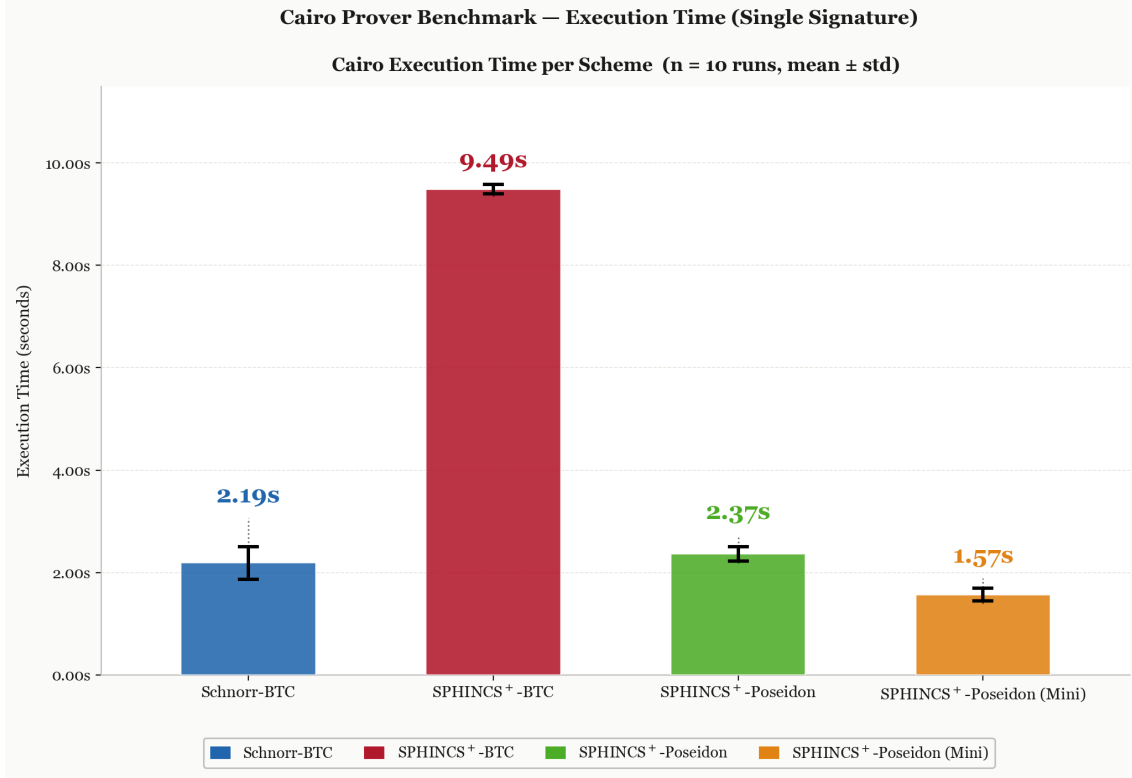


Figure 4.2: Cairo execution time for a single SPHINCS⁺ signature across hash functions and parameter sets.

parameter set/hash function pair. We note that our implementation has far fewer Cairo steps and higher prover throughput per signature, as showcased in Figure 4.2, Figure 4.3, and Figure 4.4.

It is well established in the literature that the prover speed is directly proportional to the number of Cairo cycles [26]; however, we require empirical prover throughput to determine the feasibility of SPHINCS⁺ signature aggregation. The results shown in Figure 4.3 imply that, in fact, the proof size for using Blake2s is less than its Poseidon counterparts by a significant margin, with comparable proving speed with the full Poseidon implementation. Intuitively, the SPHINCS-BTC implementation has substantially the longest execution time, which is due to its much higher execution cycles. However, its smaller proof size is counterintuitive, as that should be directly correlated with the number of execution cycles. A likely explanation is that SPHINCS-Poseidon introduces dozens of additional `Poseidon` relations during proving, increasing proof overhead, whereas SPHINCS-BTC uses fewer `Blake` relations.

A `SCHNORR-BTC` benchmark is included for comparison, representing a standard elliptic-curve digital signature algorithm. This serves as a baseline to contextualise how SNARK aggregation overhead differs between compact elliptic-curve signatures

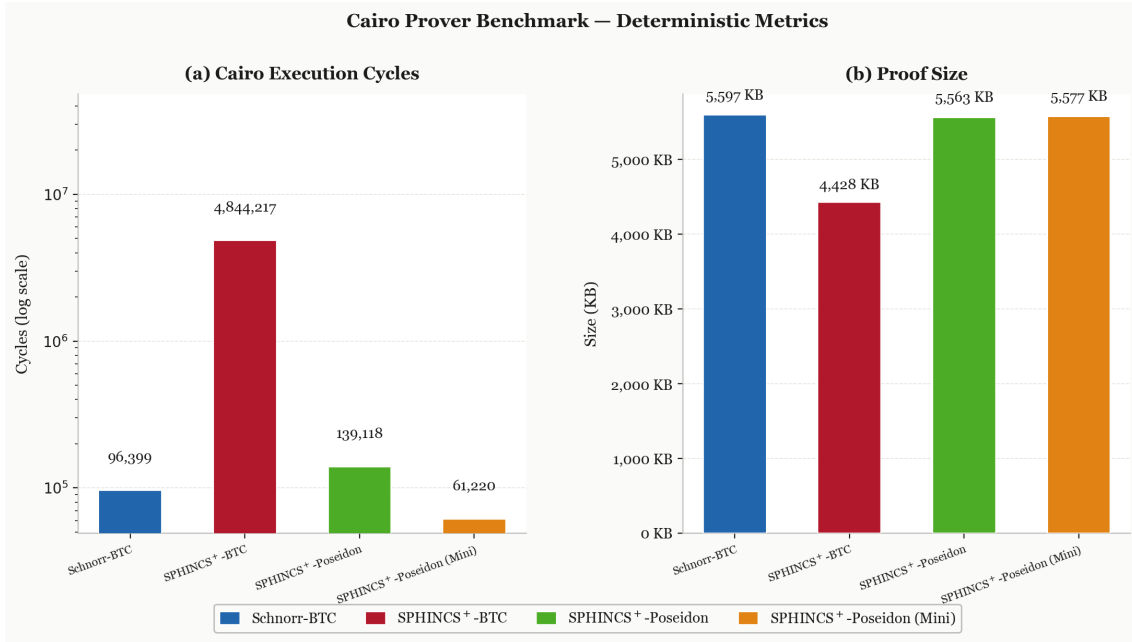


Figure 4.3: Cairo execution cycles and resulting proof size for a single SPHINCS+ signature per variant.

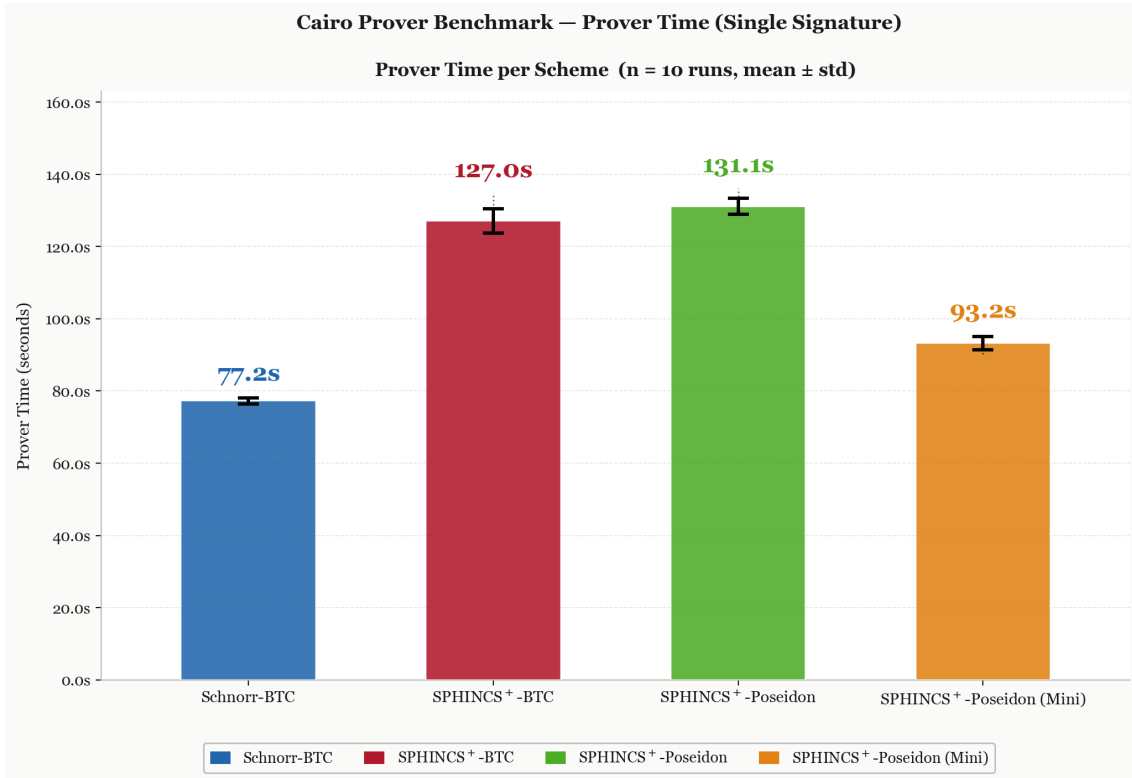


Figure 4.4: Cairo prover time for verifying a single SPHINCS+ signature across hash-function variants.

and the much larger SPHINCS+ signatures. A detailed explanation of Schnorr signatures and their implementation in Cairo can be found in Appendix A.

For further experiments, we chose to continue with the SPHINCS+ Poseidon Mini parameter set due to its strong scaling properties compared to other parameter sets and hash functions. SPHINCS-BTC offers a strong alternative but has a 44% larger proving cost despite hitting a reduced signature limit of 2^{30} . So far, the prover throughput remains the dominant constraint, which is why we prioritise this over proof size.

4.3.2 SPHINCS+ Compression Results

Carrying forward the reduced SPHINCS+ parameter set from Table 4.4, with the Poseidon hash function and a 2^{30} signature limit, this section demonstrates that a CairoVM implementation can feasibly aggregate SPHINCS+ signatures and achieve compression over naive concatenation. We note that theoretically the STARK prover has polylogarithmic scaling with respect to the number of Cairo cycles [43]; however, the initial constants are large and can be too large for feasible SPHINCS+ aggregation. We benchmark a range of SPHINCS+ signatures from $\{1, \dots, 2000\}$ in increasing powers of two and compare the prover throughput, proof sizes, and compression per SPHINCS+ signature.

As shown in Figure 4.6, a small number of SPHINCS+ signatures yield large compression ratios above 1 compared to normal concatenation. However, as represented by the proof size line, the growth in proof size is approximately polylogarithmic, and so for large signature numbers, we eventually hit the break-even point at ≈ 1600 signatures and can demonstrate 33% savings at the 2000 signature line. We achieve compression at approximately 1600 signatures, demonstrating that SNARK-based aggregation is practically feasible for thousands-signature batches.

Additionally, the graph in Figure 4.5 confirms our earlier hypothesis that execution and prover times scale almost linearly with the number of signatures, as the number of Cairo cycles increases linearly.

4.4 Experiment Conclusion

This experiment was the design-space exploration stage of the thesis: its purpose was to confirm that hash-based SNARK aggregation of SPHINCS+ is feasible at all, and to identify the hash function and parameter choices that a constraint-optimal implementation should adopt. On the first point, the results are positive. Although SPHINCS+ aggregation achieves feasible compression over naive concatenation, proof generation takes several minutes with sizes reaching several megabytes,

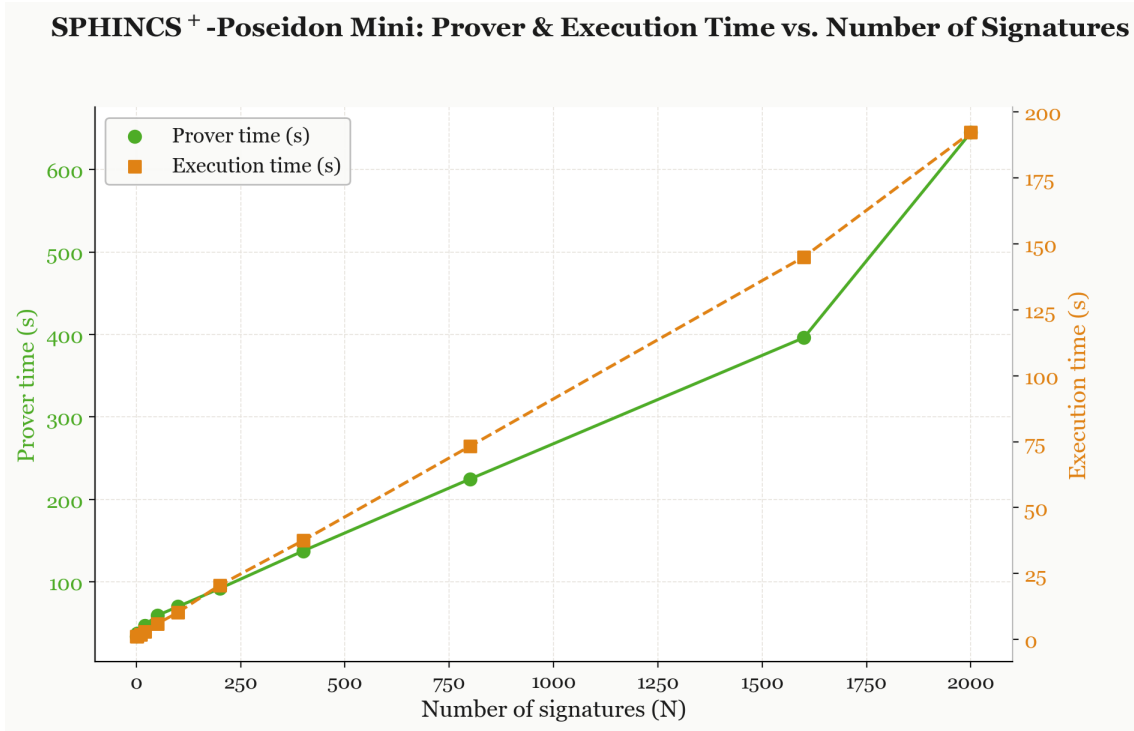


Figure 4.5: Prover and execution time vs. number of aggregated SPHINCS⁺-Poseidon Mini signatures N .

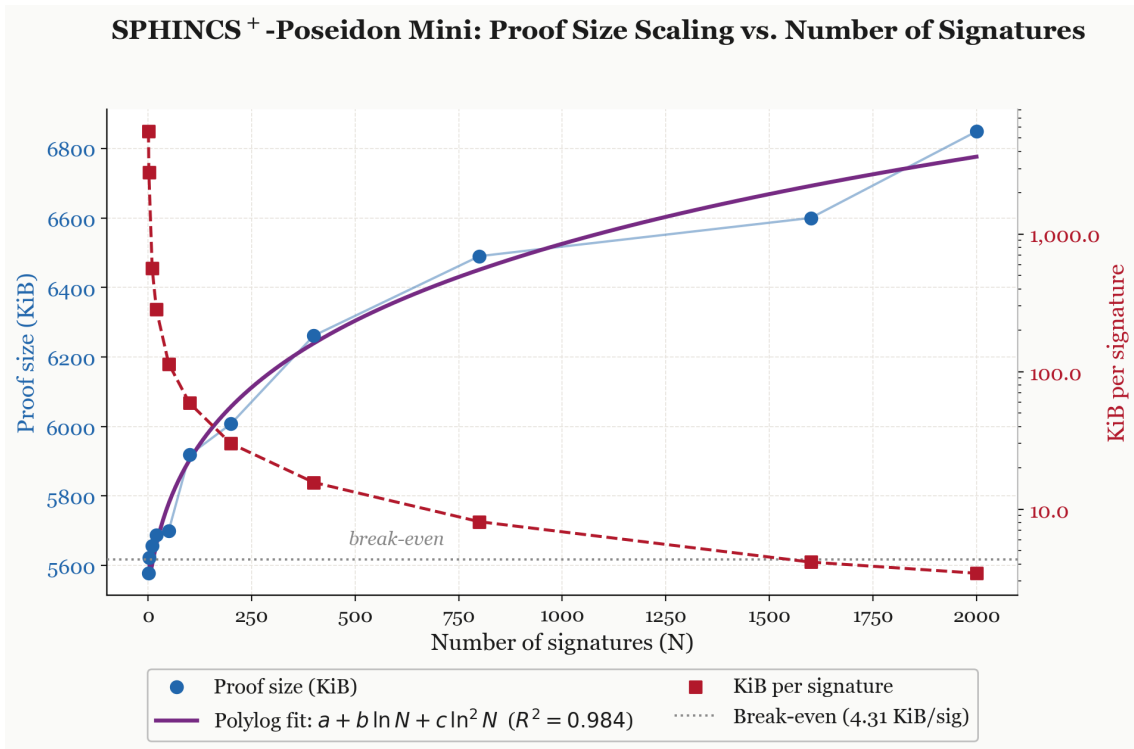


Figure 4.6: STARK proof size vs. number of aggregated SPHINCS⁺-Poseidon Mini signatures N , with polylogarithmic fit ($R^2 = 0.984$).

far exceeding practical requirements. As mentioned in Section 1.3, two-minute aggregation sits close to our one-minute target, but megabyte-scale proofs remain prohibitive. For Ethereum, real-time aggregation remains approximately $100\times$ out of reach. GPU-accelerated proving would substantially reduce prover time; however, it poses a separate problem for blockchain deployment, as any node that aggregates signatures must then own a GPU, thereby increasing the barrier to entry and increasing centralisation.

Our hypothesis of linear scaling with execution cycles was proven partially correct. Keeping the guest program fixed, increasing the number of executions does scale prover time approximately linearly; however, changing the hash primitive within the guest program has a significant impact on both proof sizes and prover times, up to 1 MB less, even with almost $100\times$ Cairo cycles as seen in Figure 4.3 and Figure 4.4. Similarly, polylogarithmic proof-size scaling holds when the guest program is fixed, and the batch size grows, but it does not hold across different guest programs.

Although Cairo is one of the few production-ready zkVMs, it has several drawbacks that could be improved. Firstly, Cairo is foremost a smart contract language for the Stark Layer 2 Blockchain [26], and so has been designed for less-intensive blockchain-like computations which can be proved within a reasonable amount of time. This is especially prevalent in the u64/u32 support that Cairo provides, which is easy for blockchain developers to reason about and use, but substantially increases costs when working with byte-intensive structures such as SPHINCS+. The second drawback is that the SNARK proofs that Cairo generates have a maximum soundness security of 96 bits. This is smaller than the 128 bits that the underlying SPHINCS+ signature provides, and so by performing SPHINCS+ aggregation with Cairo, we effectively reduce the security of SPHINCS+ to 96 bits; this is a fundamental limitation of Cairo as a proving system, not of hash-based SNARK aggregation in general. Finally, the `stwo` prover that Cairo uses to generate SNARKs relies on the Reed Solomon code proximity gap conjecture [53], which is an unproven conjecture used to reduce the size of certain SNARK proofs. Should the conjecture be proven false in the context of the `stwo` prover, the proof sizes will be larger.

However, the experiment does show a promising direction. Cairo was released in 2021, making it an early release in the field of zero-knowledge cryptography and SNARKs. Since then, several developments in SNARKs and SNARK proof generation, such as WHIR [54] and the logup lookup argument [55], have enabled orders of magnitude improvements.

Chapter 5

Experiment: Constraint-Optimal SPHINCS+ with leanVM

5.1 Hypothesis

Chapter 4 established that hash-based SNARK aggregation of SPHINCS+ is feasible, reaching a compression break-even at 1,600 signatures, but the resulting proof sizes and generation times remain orders of magnitude away from blockchain deployment. We hypothesise that the leanVM system [27], whose hash-native design is purpose-built for hash-based signature aggregation, can close most of this gap and yield a constraint-optimal baseline that approaches the targets of Section 1.3.

Carrying forward the reduced 2^{30} , $T = 304$ parameter set and the Poseidon hash function selected in Chapter 4 (Table 4.4), each signature verification requires 713 Poseidon hashes. The leanVM prover sustains approximately 600,000 Poseidon hashes per second, which gives a hashing-bound ceiling of $600,000/713 \approx 840$ signatures aggregated per second. This figure is an idealisation: it assumes Poseidon hashing dominates the cost and ignores the non-hash overhead of Merkle tree traversal and data marshalling between components. A central question of this experiment is therefore how far measured throughput falls below this ceiling, and why.

The experiment succeeds if leanVM aggregation reaches compression break-even within a low signature count and sustains a compression ratio that brings the per-signature cost close to an elliptic-curve signature, with total proof sizes under 1 MiB.

We first present implementation details and optimisations for SPHINCS+ in leanVM, then show and explain the scaling results, and finally discuss the gap between the aggregation work presented in the thesis and practical deployment.

All benchmarks are generated on an AMD EPYC 7401P 24-Core Processor.

5.2 Implementation Details

The leanVM system is a Rust crate providing a Python-like domain-specific language for writing provable guest programs. It is heavily inspired by the CairoVM paper and uses a similar architecture, with dedicated tables specialised for aggregating hash-based signatures shown in Figure 3.1. The leanVM prover also offers recursion support, enabling a single proof to attest multiple sub-proofs, thereby reducing overall proof size with minimal proof-generation overhead.

5.2.1 SPHINCS+ Bit Decomposition

Whenever we sign a message with SPHINCS+, one of the earliest steps is to generate a random value R and to produce a message digest $D = H_{msg}(R, PK.seed, PK.root, msg)$ which is then decomposed bit by bit to produce a path through the Merkle tree structure of SPHINCS+ (Figure 2.2). Classically, this is a cheap operation, but bit manipulation is much more expensive in the field arithmetic that SNARKs operate in.

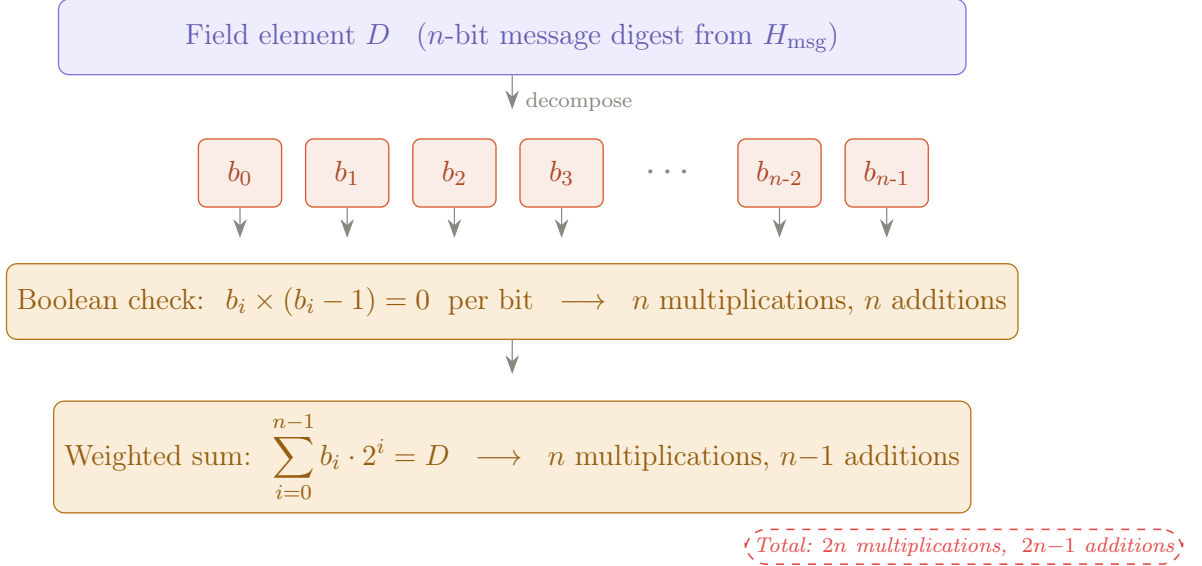
One solution for cheaper bit decomposition is to use private "witnesses" provided by the prover. For example, the prover will claim that the binary representation of the field element 5 is 101_2 . The guest program will then add two important constraints. Firstly, that each "bit" in our witness is in fact a 1 or 0, which can be ensured with the constraint $x \times (x - 1) = 0$. Secondly, we then add the bits back up in a sum $1 \times 2^0 + 0 \times 2^1 + 1 \times 2^2$ and assert the final sum is equal to 5. For an n -bit string, this requires $2n$ multiplications and $2n - 1$ sums, which adds thousands of constraints over an entire SPHINCS+ signature.

Our contribution is to instead decompose the n -bit strings into much larger chunks and use range checks on each resulting field element, which has a fixed cost of 3 cycles and perform a final reduced sum, as visualised in Figure 5.1. Note that this optimisation seems circular; we still need to split the larger chunks into bits to traverse the Merkle tree correctly, which would add more cycles and defeat the point of our current trick. However, we avoid this cost by the **JMP-range** trick described in Section 5.2.2.

5.2.2 SPHINCS+ Tree Traversal

During Merkle tree verification, we must ensure that we hash the internal-node siblings in the correct order according to the current bit path. This adds an extra if statement at each level in each Merkle tree, which increases constraints and scales linearly with AIR rows [27]. However, within leanVM, bytecode size can be scaled at sublinear cost relative to prover throughput [27]. For that reason, we can use a

(a) Naïve bit decomposition



(b) Optimised chunk decomposition

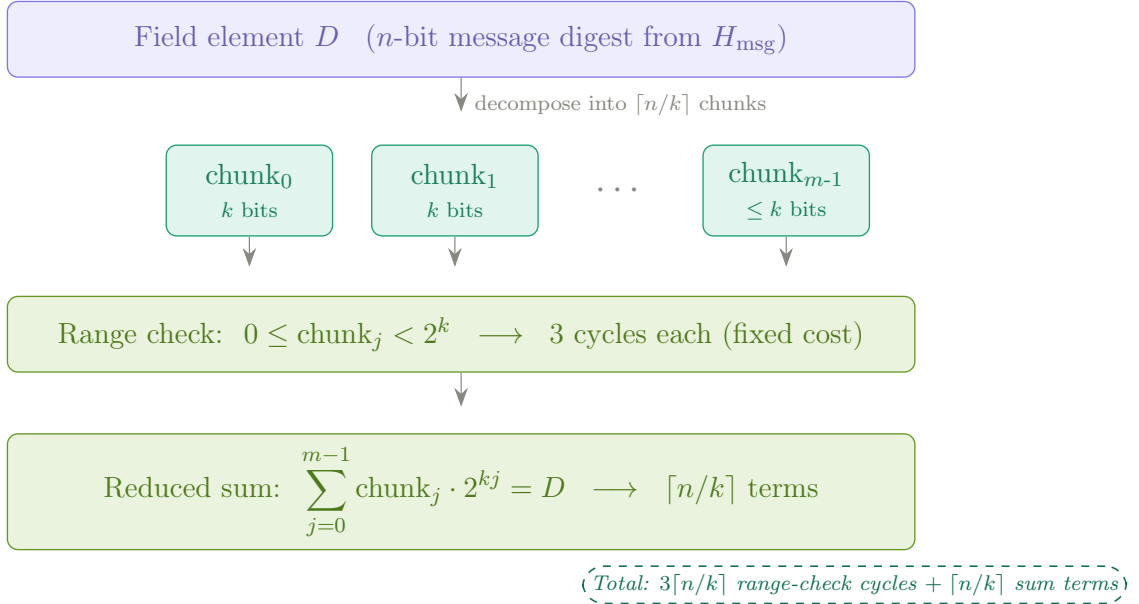


Figure 5.1: Optimised leanVM bit decomposition, replacing per-bit constraints with chunked range checks and a reduced sum.

JMP-range trick, which inflates the bytecode size but incurs almost no cost to the prover, since the bytecode is public information. The pseudocode for Merkle tree authentication is given in Algorithm 1, which contains d internal if statements during the path authentication. Each if statement will cost at least one JMP instruction.

Algorithm 1 Merkle Tree Path Authentication

Require: Leaf value v , leaf index i , authentication path $\pi = (\pi_1, \pi_2, \dots, \pi_n)$, hash function H , tree depth n

Returns: Merkle tree root computed according to authentication path π

```

1:  $currentHash \leftarrow v$ 
2:  $b \leftarrow i_2$  ▷ Convert  $i$  into an  $n$ -bit string  $b$ 
3: for  $k \leftarrow 1$  to  $n$  do
4:   if  $b[k] = 0$  then ▷ Current node is a left child
5:      $currentHash \leftarrow H(currentHash \parallel \pi_k)$ 
6:   else ▷ Current node is a right child
7:      $currentHash \leftarrow H(\pi_k \parallel currentHash)$ 
8:   end if
9: end for
10: return  $currentHash$ 

```

For an n -bit string, we will have n IF statements which translate to n conditional JMP calls during execution. However, if n is small enough, we can instead specialise 2^n different functions of Algorithm 1 whose IF statements have been collapsed during compile-time as shown in Algorithm 2.

Algorithm 2 Merkle Tree Path Authentication Specialised with $i = 5$

Require: Leaf value v , authentication path $\pi = (\pi_1, \pi_2, \pi_3)$, hash function H

Returns: Merkle tree root computed according to authentication path π

```

1:  $currentHash \leftarrow v$  ▷  $i_2 = 101$ 
2:  $currentHash \leftarrow H(\pi_k \parallel currentHash)$  ▷ Current node is a right child
3:  $currentHash \leftarrow H(currentHash \parallel \pi_k)$  ▷ Current node is a left child
4:  $currentHash \leftarrow H(\pi_k \parallel currentHash)$  ▷ Current node is a right child
5: return  $currentHash$ 

```

This requires the compiler to generate 2^n specialised functions, which are placed consecutively in the bytecode. Before calling Merkle tree authentication, we perform a conditional jump to the correct address, calculated via a linear function. During execution, we have a single JMP instruction that incurs an overhead of only 1 cycle [27]. This inflates bytecode size, but the prover's work is independent of bytecode length. Instead, the prover commits to bytecode accesses and verifies instruction consistency, which scales with execution cycles, not bytecode size.

This trick is especially useful when combined with the chunk trick described above in Section 5.2.1. If we split a large bitstring into chunks of 5 bits, we can

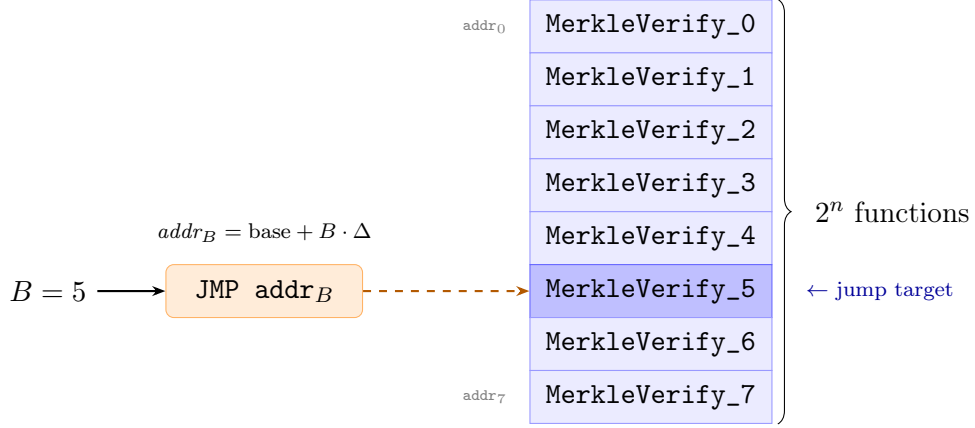


Figure 5.2: JMP-range trick: an affine JMP into 2^n consecutive specialised functions, replacing n branches with one cycle.

specialise 2^5 functions which are dependent on the chunk values and substantially reduce the number of internal IF statements.

5.3 Results

We benchmarked a SPHINCS+ implementation using the reduced 2^{30} , $T = 304$ parameter set and the Poseidon hash function as described in Table 4.4. We measure the prover throughput alongside the proof size as we increase the number of signatures aggregated.

N	Total proof	Bytes / sig	Compression
1	257 KiB	263,232	$0.02\times$
4	272 KiB	69,696	$0.06\times$
16	267 KiB	17,088	$0.26\times$
64	273 KiB	4,368	$1.01\times$
256	330 KiB	1,320	$3.34\times$
512	365 KiB	730	$6.04\times$
1,024	416 KiB	416	$10.61\times$
2,500	530 KiB	217	$20.31\times$
<i>Reference baselines (no aggregation):</i>			
ECDSA (Ethereum)	—	65	—
SPHINCS ⁺ -Poseidon	—	4,412	—

Table 5.1: SPHINCS⁺-Poseidon aggregation: per-signature proof cost and compression ratio.

As hypothesised, prover throughput grows nearly linearly with signature count (Figure 5.3), with a small discontinuity at 2048 to 2500 signatures. We attribute this to leanVM’s power-of-two alignment, which introduces discrete performance

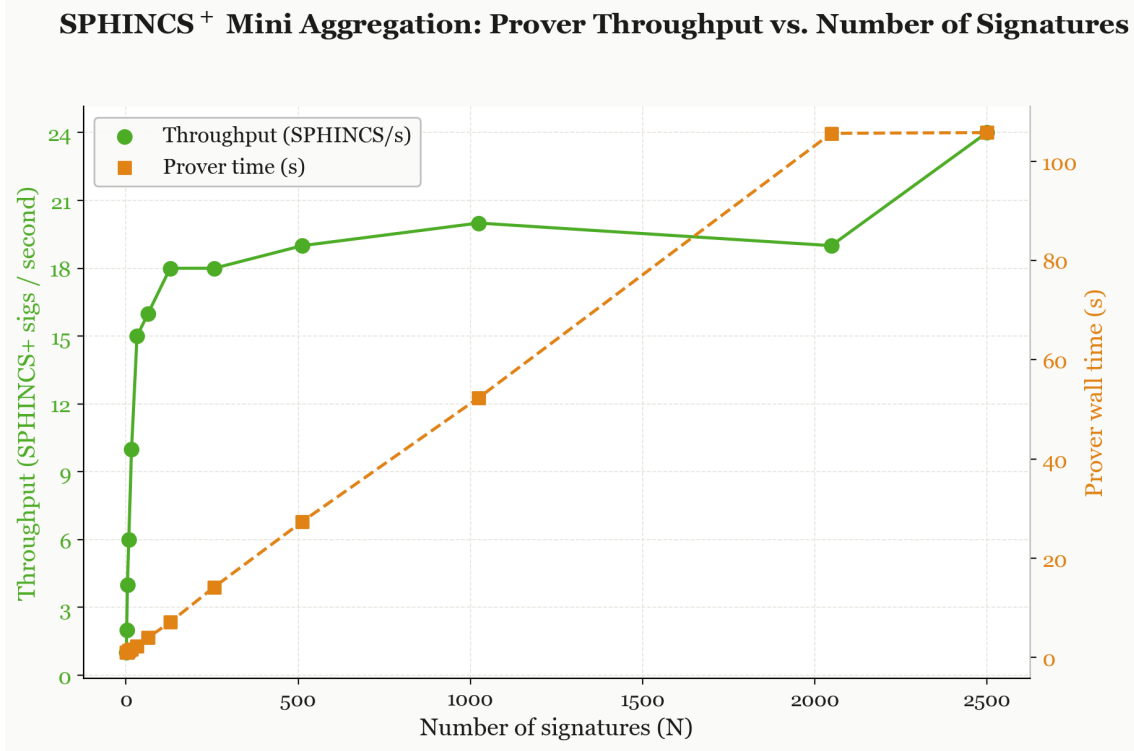


Figure 5.3: SPHINCS+ Mini prover throughput against number of signatures aggregated using leanVM

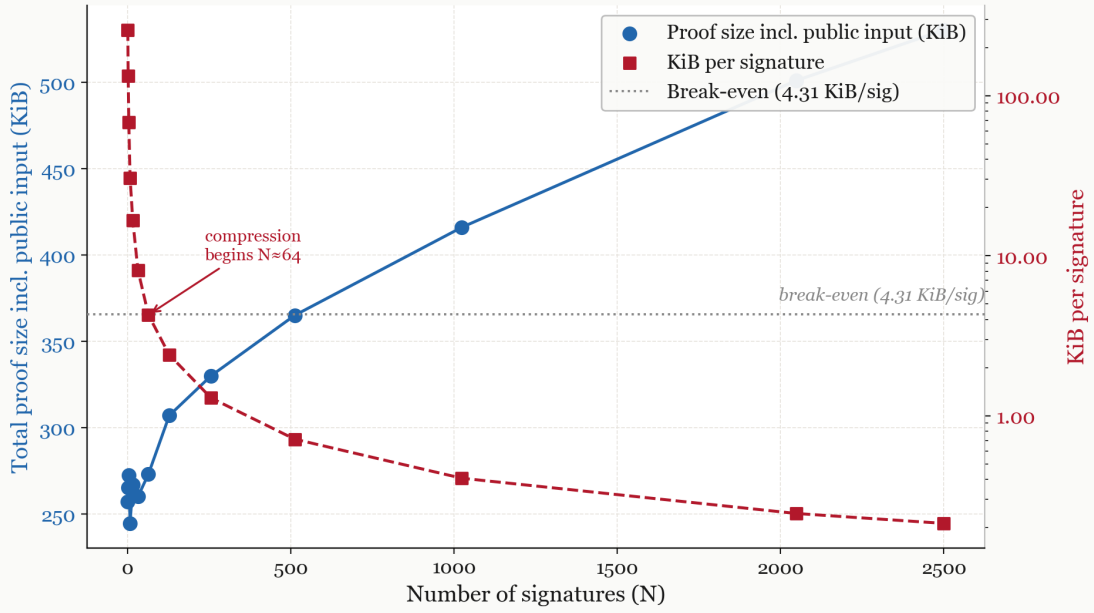
jumps; leanVM pads all table lengths to the nearest power of two, but if signatures are added without crossing a power-of-two boundary, proof time remains almost unchanged, as demonstrated.

Notably, throughput per signature remains consistent at approximately 20 SPHINCS+ signatures per second, which implies that we are compute-bound; switching to more modern hardware or GPUs could increase throughput by orders of magnitude. For this experiment, it is sufficient to demonstrate feasible speeds on commodity hardware.

5.3.1 Higher-Throughput Results on Cloud Hardware

To provide an upper bound on prover throughput with more capable hardware, we ran a subset of benchmarks on a Hetzner AX42-U server. At 320 signatures, the prover achieved 140 SPHINCS+ signatures per second, a $7\times$ improvement over the AMD EPYC results. However, at 512 signatures the throughput dropped to 50 signatures per second. This is a direct consequence of the Poseidon table power-of-two alignment described above: the jump from 320 to 512 signatures forces leanVM to pad the Poseidon table to the next power of two, nearly doubling the effective table size and substantially increasing prover cost. The two data points together

SPHINCS⁺ Mini Aggregation: Proof Size & Compression vs. Number of Signatures



Public input: 16 field elements (64 bytes) per signature added to each proof size.

Figure 5.4: leanVM aggregate proof size and compression ratio against the number of aggregated SPHINCS⁺ Mini signatures.

illustrate that peak throughput is sensitive to where a given batch size falls relative to the nearest power-of-two boundary, and that the 140 signatures per second figure represents a near-optimal alignment rather than a sustained average rate.

With leanVM, proof compression is highly efficient, with a break-even point at 64 signatures. As we scale the number of signatures further, we reach a minimum of 217 bytes per signature when aggregating 2,500 signatures, less than four times the 65-byte elliptic curve signature [15]. This is a compression ratio of 20 $\times$ and would feasibly allow thousands of transactions to be included in a blockchain block for under 1 MB.

5.4 Experiment Conclusion

Building on the reduced parameter set and Poseidon hash function selected in Chapter 4, this experiment set out to turn that feasibility result into a constraint-optimal baseline that approaches the deployment targets of Section 1.3. The leanVM prover achieves strong proof compression: a ratio of over $20\times$ with total proof sizes of approximately 500 KiB, well within the target of 1 MiB.

As the logarithmic curve in Figure 5.4 suggests, the compression ratio should only improve further. It is plausible that the per-signature proof size could approach that of elliptic curve signatures. The main cost of reaching such thresholds is twofold. The prover throughput, which is already the dominant constraint, will continue to grow linearly and must be further reduced with better hardware or improved zkVM implementations. Secondly, the amount of memory required to prove the aggregation of thousands or tens of thousands of transactions can rise to the order of hundreds of gigabytes, which is infeasible for any consumer hardware which may want to take part in signature aggregation.

The results show that the hashing-bound ceiling of approximately 840 SPHINCS+ signatures per second remains some distance from current benchmarks, even with the optimised leanVM prover. This confirms the hypothesis that the ceiling is an idealisation: even with the optimisations described in Section 5.2, the dominant overhead in SPHINCS+ aggregation is not Poseidon hashing but Merkle tree traversal and the associated data marshalling between components. Although fixing this problem is achievable by using dedicated "precompiles" where specific SPHINCS+ operations are embedded as extra tables in the leanVM architecture, this will substantially increase the complexity of the leanVM prover, increasing the surface area for soundness bugs and mistakes.

This implementation is experimental, not production-ready. The SPHINCS+ guest program uses simplified domain separation because leanVM does not yet provide optimised support for full tweaks: the way its Poseidon hashing interface is exposed makes full-tweak hashing disproportionately expensive. Benchmarking full tweaks would therefore reflect this interface limitation rather than the true performance of SPHINCS+ aggregation, so we defer it until leanVM gains native support. A production version would require full domain separation, which would increase the number of cycles required and reduce prover throughput accordingly. Additionally, leanVM currently supports only 124 bits of soundness rather than the target 128 bits [27], and the architecture remains under active development with ongoing changes.

Chapter 6

Discussion

As discussed in Section 1.3, this chapter examines the gap between the experimental results and the practical deployment of SNARK-based aggregation of SPHINCS+. Although the experiments demonstrate large compression ratios, several key milestones remain before SNARK aggregation can be used in Bitcoin or Ethereum.

6.1 SNARK-based aggregation in Bitcoin

With respect to prover throughput and proof sizes, we can aggregate thousands of signatures well within 1 MiB, requiring approximately 2 minutes (see Figure 5.3) to aggregate the thousands of signatures a Bitcoin block requires [19]. On more capable hardware, up to 140 signatures per second is achievable; this figure depends critically on Poseidon table power-of-two alignment (see Section 5.3.1). These results approach Bitcoin’s aggregation requirements, but a critical obstacle remains.

The implementation in leanVM is based entirely on the Poseidon hash function. This would require Bitcoin to merge the Poseidon hash function as a cryptographic primitive in order to accept the current hash-based SNARK implementation. However, Poseidon (released 2021 [4]) has received far less scrutiny than its more mature siblings, such as the SHA [30] or Blake2 [10] families of hash functions. Poseidon2 [56] has suffered recent attacks [57], raising concerns about Poseidon’s security for Bitcoin adoption. Bitcoin has a more conservative approach to security than other blockchains, such as Ethereum. Paired with the fact that Poseidon is substantially slower to execute outside of a zkVM (Table 4.1), this makes Poseidon unsuitable for Bitcoin adoption.

Instead, Bitcoin would require all SNARK aggregation for SPHINCS+ to be entirely reliant on SHA-256, which is substantially less efficient in SNARKs compared to Poseidon (Table 4.3).

6.2 SNARK-based aggregation in Ethereum

As discussed in Section 1.2.3, several practical scenarios exist where SNARK aggregation of SPHINCS+ would be valuable within a post-quantum Ethereum.

Unlike Bitcoin, Ethereum is more tolerant of the use of the Poseidon function [27]; however, the aggregation performance still has a significant gap to reach the millisecond prover times that Ethereum requires to aggregate hundreds of signatures per second [58]. Additionally, the proof sizes must be less than 100 KiB for Ethereum to achieve the same block sizes as it currently has [23]; this can be achieved with recursion, which we discuss as part of the future work.

Our results also let us draw a precise boundary between SPHINCS+ and its stateful sibling XMSS, the distinction we flagged in Section 1.2.3. The deciding factor is statefulness. Within consensus, each validator must sign exactly one message per slot, so the slot itself supplies the monotonic state that XMSS requires and rules out the catastrophic key-reuse that statelessness is designed to avoid [27]. In this setting XMSS is the stronger choice: its simpler structure yields faster aggregation and smaller proofs than SPHINCS+ [24]. SPHINCS+ aggregation is therefore not the right tool for the consensus layer, and identifying this cleanly is itself a contribution rather than a shortcoming of the scheme.

The transaction-signature setting falls on the other side of this boundary. Ordinary users and wallets cannot be relied upon to maintain state across crashes or device changes, which excludes XMSS and leaves SPHINCS+ as the only standardised hash-based option [27]. This is precisely the setting our contribution targets, and it is the setting Ethereum is itself pursuing SPHINCS+ aggregation for, which offers a strong basis for future research [27].

6.3 Comparison with LaBRADOR Falcon Aggregation

Section 2.4.5 introduces the Falcon post-quantum signature scheme and its aggregation with LaBRADOR. LaBRADOR achieves compression at approximately 110 signatures, while our scheme achieves compression after 64 signatures. Årdal et al. [51] demonstrate aggregation of 500 Falcon signatures with 121 bits of security on the order of ≈ 100 KiB, which is substantially smaller than the sizes reported in Table 5.3. Falcon signatures are substantially smaller at 666 bytes [51] than the 4,412 bytes of our reduced SPHINCS+ variant.

Chapter 7

Conclusion

The eventual arrival of cryptographically-relevant quantum computers threatens the elliptic-curve signatures that secure Bitcoin and Ethereum, since Shor’s algorithm breaks them in polynomial time. Of the signature schemes standardised by NIST in response, the hash-based SPHINCS+ is the most conservative choice for blockchains: it rests only on the hash functions these systems already trust, and it is the sole standardised stateless scheme, making it the only candidate suited to the transaction-signature setting where signers cannot maintain reliable state. SPHINCS+ carries one decisive drawback. Its signatures are roughly 120 times larger than the elliptic-curve signatures they would replace, and unlike BLS it has no algebraic structure that permits native aggregation. This is the dilemma at the heart of the thesis: the most trustworthy post-quantum signature is also the least deployable, and bridging that gap requires aggregation external to the signature itself. SNARKs supply exactly this, provided they are hash-based so that aggregation does not reintroduce the quantum vulnerability SPHINCS+ was chosen to avoid.

This thesis made three contributions towards closing that gap. First, we used CairoVM to explore the design space of hash-based SPHINCS+ aggregation, quantifying the cost of different hash functions and parameter sets. Second, we built a constraint-optimal prototype on the Ethereum Foundation’s leanVM, applying field-native optimisations to establish a state-of-the-art performance baseline. Third, we used these benchmarks to draw the boundary of where SPHINCS+ aggregation belongs and to quantify the gap that remains before practical deployment, measured against the targets that aggregation must meet: on the order of 1 minute for Bitcoin and 1 second for Ethereum, with aggregation sizes on the order of 100 KiB and 1 MiB respectively.

Focusing first on CairoVM, we calculated the cost of different hash functions and parameters within SPHINCS+. Our results showed that the Poseidon hash function is 100 times cheaper with respect to execution cycles but matches Blake2s in proof size and throughput. We hypothesised that SPHINCS+ aggregation was

feasible with hash-based SNARKs and demonstrated a compression break-even at 1,600 signatures, with 33% compression at 2,000 signatures.

Using the reduced SPHINCS+ variant and Poseidon hash function established with CairoVM, we moved to the Ethereum Foundation’s in-house leanVM for a constraint-optimal prototype SPHINCS+ implementation. We applied several zk-specific optimisations such as **JMP-range** to reduce the SPHINCS+ guest program cost and achieved compression at 64 signatures; this is a better break-even point than LaBRADOR’s approximately 110 signatures, demonstrating that hash-based SNARK aggregation is competitive with lattice-based alternatives. We achieved aggregation rates of approximately 20 signatures per second on test hardware, with up to 140 signatures per second measured on a higher-core-count cloud server at well-aligned batch sizes, and a maximum compression ratio of $20\times$.

Finally, we discussed the gap between the deployment of SNARK aggregation in Bitcoin and Ethereum, and drew a clear boundary for where SPHINCS+ aggregation belongs: the stateless transaction-signature layer, where statelessness is mandatory, rather than consensus, where the slot supplies the state that makes XMSS the stronger choice. Current benchmarks are not yet sufficient for practical deployment; nevertheless, hash-based SNARK aggregation of SPHINCS+ is a promising scaling solution for post-quantum transaction signatures, and this thesis establishes a state-of-the-art baseline.

7.1 Future Work

7.1.1 SPHINCS+ aggregation with SHA

As discussed in Section 6.1, the Poseidon hash function has several disadvantages for its use in Bitcoin, while the SHA-256 variant benchmarked in Section 4 was prohibitively expensive to run on normal consumer hardware. A natural step for SNARK-based aggregation is to explore optimised SHA-256 SNARK implementations. SHA-256 lends itself very well to look-up arguments such as LogUp [55] or Plookup [46] due to its byte-based structure [30]. Many alternative zkVMs, such as RISC0 [59], include specialised SHA-256 extensions which can be explored. GPU-accelerated proving significantly increases prover throughput in zkVMs [60] and offers an attractive solution for SPHINCS+ aggregation with SHA-256 in order to reach feasible real-time proving.

7.1.2 NIST-approved SPHINCS+ parameter set

NIST has recently standardised a reduced parameter set for SPHINCS+, which offers a limit of 2^{24} signatures [61], requiring fewer than 300 hashes per signature verification. Implementing this reduced parameter set would demonstrate even smaller proof sizes and faster proof throughput, while a signature limit of 2^{24} remains more than sufficient for most users in a blockchain. As discussed in Section 5.2, for zkVMs with optimised Poseidon implementations, the main bottleneck arises from byte manipulation and memory copying, though this will decrease as the number of verification hashes reduces.

This NIST-approved parameter set does have significant drawbacks. It has a single Merkle tree layer with a depth of 22 [61], which will substantially increase key generation time and signing costs for such a small parameter set. These are all statistics which can be further explored and tested to check whether NIST’s reduced parameter set is suitable for Bitcoin and Ethereum.

7.1.3 Recursive proof aggregation

A key limitation identified above is that the leanVM implementation produces proof sizes of around 500 KiB, which is $5\times$ too large for aggregating post-quantum transaction signatures. However, leanVM supports recursive proof aggregation [27], which can reduce the total proof size with only a small overhead in prover throughput and proof verification. The leanVM implementation of XMSS can recursively aggregate proofs into 100 KiB [27]. As recursive proof sizes are much more compact compared to their child proofs, recursion for SPHINCS+ offers a viable route forward.

7.1.4 Deployment within Blockchains

This thesis is scoped to the internal performance of SPHINCS+ aggregation and does not address how the scheme would enter a live protocol. Two paths exist. SPHINCS+ could be enshrined as a direct replacement for ECDSA, giving uniform post-quantum security at the cost of forcing every user onto one scheme with fixed parameters. Alternatively, Ethereum could expose SPHINCS+ through account abstraction [62], letting users opt in to a verifier expressed in the leanVM instruction set; this avoids a hard fork but complicates aggregation, as a single block may now mix SPHINCS+ with other schemes.

Either path leaves open who performs aggregation. Our benchmarks show commodity hardware is sufficient for the work itself (Section 5.3.1), so the open question is timeliness: aggregating the thousands of signatures per block within the latency budget of Section 1.3 may still favour a dedicated aggregator. On Ethereum this could fold into the proposer/builder pipeline; Bitcoin would plausibly delegate it to miners, though a new participant may be required. The dominant costs, however, lie not in who aggregates but in the aggregation itself: proof verification is cheap, so it is the aggregation throughput against the per-block latency budget and the resulting proof size that gate deployment. These, together with the aggregator incentives and liveness concerns raised in Section 1.2.3, must be resolved first.

Bibliography

- [1] Stefan Kölbl and Jade Philipoom. A note on sphincs+ parameter sets. *Cryptology ePrint Archive*, 2022.
- [2] Mikhail Kudinov and Jonas Nick. Hash-based signature schemes for bitcoin. *Cryptology ePrint Archive*, 2025.
- [3] Lorenzo Grassi, Dmitry Khovratovich, Reinhard Lüftenegger, Christian Rechberger, Markus Schofnegger, and Roman Walch. Reinforced concrete: A fast hash function for verifiable computation. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, pages 1323–1335, 2022.
- [4] Lorenzo Grassi, Dmitry Khovratovich, Christian Rechberger, Arnab Roy, and Markus Schofnegger. Poseidon: A new hash function for {Zero-Knowledge} proof systems. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 519–535, 2021.
- [5] Peter W Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM review*, 41(2):303–332, 1999.
- [6] David Cooper et al. Stateless hash-based digital signature standard. 2024.
- [7] National Institute of Standards and Technology. Module-lattice-based digital signature standard. Federal information processing standards publication (fips) 204, U.S. Department of Commerce, National Institute of Standards and Technology, Gaithersburg, MD, 2024.
- [8] Google. A clear-eyed approach to our post-quantum cryptography migration. Google Security Blog, 2025. Available: <https://blog.google/innovation-and-ai/technology/safety-security/cryptography-migration-timeline/>.
- [9] Cloudflare. The post-quantum cryptography roadmap. The Cloudflare Blog, 2025. Available: <https://blog.cloudflare.com/post-quantum-roadmap/>.

- [10] Jean-Philippe Aumasson, Willi Meier, Raphael C-W Phan, and Luca Henzen. The hash function blake. 2014.
- [11] Luca De Feo, David Kohel, Antonin Leroux, Christophe Petit, and Benjamin Wesolowski. Sqisign: compact post-quantum signatures from quaternions and isogenies. In *International conference on the theory and application of cryptology and information security*, pages 64–93. Springer, 2020.
- [12] Johannes Buchmann, Erik Dahmen, and Andreas Hülsing. Xmss-a practical forward secure signature scheme based on minimal security assumptions. In *International Workshop on Post-Quantum Cryptography*, pages 117–129. Springer, 2011.
- [13] David McGrew, Panos Kampanakis, Scott Fluhrer, Stefan-Lukas Gazdag, Denis Butin, and Johannes Buchmann. State management for hash-based signatures. In *International Conference on Research in Security Standardisation*, pages 244–260. Springer, 2016.
- [14] Dan Boneh, Craig Gentry, Ben Lynn, and Hovav Shacham. Aggregate and verifiably encrypted signatures from bilinear maps. In *International conference on the theory and applications of cryptographic techniques*, pages 416–432. Springer, 2003.
- [15] Gavin Wood et al. Ethereum: A secure decentralised generalised transaction ledger. *Ethereum project yellow paper*, 151(2014):1–32, 2014.
- [16] Robert Campbell. Hybrid post-quantum signatures for bitcoin and ethereum: A protocol-level integration strategy. *The Journal of The British Blockchain Association*, 2025.
- [17] Thomas Coratger and Srinath Setty. Post quantum signature aggregation: a folding approach. <https://ethresear.ch/t/post-quantum-signature-aggregation-a-folding-approach/23639>, December 2025. Accessed: 2026-05-06.
- [18] Aniket Kate, Gregory M Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In *International conference on the theory and application of cryptology and information security*, pages 177–194. Springer, 2010.
- [19] Blockchain.com. Average no. transaction per block, 2025.
- [20] Blockchain.com. Median confirmation time, 2025.

- [21] Blockchain.com. Average block size (mb), 2025.
- [22] YCharts. Ethereum average block time (daily) — historical data & trends, 2026. Data sourced from Etherscan. Last value: 12.04 seconds (Mar 14, 2026).
- [23] Etherscan. Ethereum average block size chart, 2026. Historical average block size in bytes of the Ethereum blockchain.
- [24] Justin Drake, Dmitry Khovratovich, Mikhail Kudinov, and Benedikt Wagner. Hash-based multi-signatures for post-quantum ethereum. *Cryptology ePrint Archive*, 2025.
- [25] Ward Beullens and Gregor Seiler. Labrador: compact proofs for r1cs from module-sis. In *Annual International Cryptology Conference*, pages 518–548. Springer, 2023.
- [26] Lior Goldberg, Shahar Papini, and Michael Riabzev. Cairo—a turing-complete stark-friendly cpu architecture. *Cryptology ePrint Archive*, 2021.
- [27] Emile and Thomas Coratger. leanvm. https://github.com/leanEthereum/leanVM/releases/download/spec-latest/minimal_zkVM.pdf, 2026. Accessed: 2026-05-06.
- [28] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system. 2008.
- [29] Don Johnson, Alfred Menezes, and Scott Vanstone. The elliptic curve digital signature algorithm (ecdsa). *International journal of information security*, 1(1):36–63, 2001.
- [30] National Institute of Standards and Technology. Secure hash standard (shs)(fips pub 180–4). *Fed Inf Process Stand Publ*, 180:36, 2015.
- [31] Lov K Grover. A fast quantum mechanical algorithm for database search. In *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, pages 212–219, 1996.
- [32] Matthew Campagna, Lidong Chen, Ozgür Dagdelen, Jintai Ding, J Fernick, Nicolas Gisin, Donald Hayford, Thomas Jennewein, Norbert Lütkenhaus, Michele Mosca, et al. Quantum safe cryptography and security: An introduction, benefits, enablers and challenges. *European Telecommunications Standards Institute*, 8:1–64, 2015.
- [33] Vlad Gheorghiu and Michele Mosca. Benchmarking the quantum cryptanalysis of symmetric, public-key and hash-based cryptographic schemes. *arXiv preprint arXiv:1902.02332*, 2019.

- [34] Project Eleven. Bitcoin risq list, 2024.
- [35] Johannes Buchmann, Erik Dahmen, Sarah Ereth, Andreas Hülsing, and Markus Rückert. On the security of the winternitz one-time signature scheme. *International Journal of Applied Cryptography*, 3(1):84–96, 2013.
- [36] David A Cooper, Daniel C Apon, Quynh H Dang, Michael S Davidson, Morris J Dworkin, Carl A Miller, et al. Recommendation for stateful hash-based signature schemes. *NIST Special Publication*, 800(208):800–208, 2020.
- [37] Andreas Hülsing, Lea Rausch, and Johannes Buchmann. Optimal parameters for xmss mt. In *International conference on availability, reliability, and security*, pages 194–208. Springer, 2013.
- [38] Alfred Menezes. Hash-based signature schemes. <https://cryptography101.ca/hash-based-signature-schemes/>, June 2025. Accessed: 2026-01-12.
- [39] Jean-Philippe Aumasson, Daniel J Bernstein, Ward Beullens, Christoph Dobraunig, Maria Eichlseder, Scott Fluhrer, Stefan-Lukas Gazdag, Andreas Hülsing, Panos Kampanakis, Stefan Kölbl, et al. Sphincs. Technical report, Technical report, Stanford Univ., Tech. Rep, 2019.
- [40] National Institute of Standards and Technology. Additional SLH-DSA Parameter Sets for Limited-Signature Use Cases. Technical report, U.S. Department of Commerce, National Institute of Standards and Technology, Gaithersburg, MD, 2026.
- [41] Quantum Canary. Quantum threats and blocksize wars: Can bitcoin stay secure? <https://www.quantumcanary.org/insights/quantum-threats-and-blocksize-wars-can-bitcoin-stay-secure>, 2024. Accessed: 2026-01-14.
- [42] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *Conference on the theory and application of cryptographic techniques*, pages 186–194. Springer, 1986.
- [43] Alan Szepieniec. Anatomy of a stark, part 1: Stark overview, 2025.
- [44] Ariel Gabizon, Zachary J Williamson, and Oana Ciobotaru. Plonk: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. *Cryptology ePrint Archive*, 2019.
- [45] Jens Groth. On the size of pairing-based non-interactive arguments. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 305–326. Springer, 2016.

- [46] Ariel Gabizon and Zachary J Williamson. plookup: A simplified polynomial protocol for lookup tables. *Cryptology ePrint Archive*, 2020.
- [47] Alan Szepieniec, Tomer Ashur, and Siemen Dhooghe. Rescue-prime: a standard specification (sok). *Cryptology ePrint Archive*, 2020.
- [48] Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Scalable, transparent, and post-quantum secure computational integrity. *Cryptology ePrint Archive*, 2018.
- [49] Ulrich Haböck. A summary on the fri low degree test. *Cryptology ePrint Archive*, 2022.
- [50] Jeremy Bruestle, Paul Gafni, and the RISC Zero Team. Risc zero zkvm: Scalable, transparent arguments of risc-v integrity. Technical report, RISC Zero, 2023. Draft.
- [51] Marius A Aardal, Diego F Aranha, Katharina Boudgoust, Sebastian Kolby, and Akira Takahashi. Aggregating falcon signatures with labrador. In *Annual International Cryptology Conference*, pages 71–106. Springer, 2024.
- [52] Abdel. Quantum-resistant signatures for bitcoin: A STARK experiment. <https://hackmd.io/@AbdelStark/pq-btc-sig-stark>, 2025. Accessed: 2025-05-16.
- [53] Eli Ben-Sasson, Dan Carmon, Yuval Ishai, Swastik Kopparty, and Shubhangi Saraf. Proximity gaps for reed–solomon codes. *Journal of the ACM*, 70(5):1–57, 2023.
- [54] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. Whir: Reed–solomon proximity testing with super-fast verification. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 214–243. Springer, 2025.
- [55] Lev Soukhanov. Logup*: faster, cheaper logup argument for small-table indexed lookups. *Cryptology ePrint Archive*, 2025.
- [56] Lorenzo Grassi, Dmitry Khovratovich, and Markus Schofnegger. Poseidon2: A faster version of the poseidon hash function. In *International Conference on Cryptology in Africa*, pages 177–203. Springer, 2023.
- [57] Simon-Philipp Merz and Àlex Rodríguez García. Skipping class: Algebraic attacks exploiting weak matrices and operation modes of poseidon2 (b). *Cryptology ePrint Archive*, 2026.

- [58] Etherscan. Ethereum daily transactions chart, 2026. Ethereum block explorer and analytics platform.
- [59] Jeremy Bruestle, Paul Gafni, et al. Risc zero zkvm: scalable, transparent arguments of risc-v integrity. *Draft. July, 29, 2023.*
- [60] Tarunesh Verma, Yichao Yuan, Nishil Talati, and Todd Austin. Zkprophet: Understanding performance of zero-knowledge proofs on gpus. In *2025 IEEE International Symposium on Workload Characterization (IISWC)*, pages 142–154. IEEE, 2025.
- [61] Dustin Moody and Quynh Dang. Additional slh-dsa parameter sets for limited signature use cases. Technical report, National Institute of Standards and Technology, 2026.
- [62] Vitalik Buterin, Yoav Weiss, Dror Tirosh, Shahaf Nacson, Alex Forshtat, Kristof Gazso, and Tjaden Hess. ERC-4337: Account Abstraction Using Alt Mempool. Ethereum Improvement Proposals, no. 4337, September 2021. Available: <https://eips.ethereum.org/EIPS/eip-4337>.
- [63] Claus P Schnorr. Method for identifying subscribers and for generating and verifying electronic signatures in a data exchange system, February 19 1991. US Patent 4,995,082.

Chapter 8

Declarations

8.1 Use of Generative AI

I acknowledge the use of Claude Code (models: Sonnet 4.6, Opus 4.8, by Anthropic, <https://claude.ai/new>) to assist with implementation architecture planning and proofreading. These tools were used to help generate figures for the report from raw table data and to explore the leanVM codebase to answer questions on the underlying architecture. I confirm that no AI-generated content has been presented as my own work.

8.2 Ethical Considerations

This work includes cryptographic implementations, which are notoriously responsible for more vulnerabilities in the wild than theoretical or academic attacks. The work presented in this thesis should not be used within production scenarios without extensive security testing and peer review. However, the thesis presents experimental work for benchmarking purposes and is not presented as a mature implementation. For that reason, negative societal impacts are minimal.

Additionally, we did not use any private data or sensitive data. All signatures were generated randomly as part of the benchmark.

8.3 Sustainability

This work required minimal resources; most development was performed on a consumer laptop with benchmarks generated on an AMD EPYC 7401P 24-Core Processor. Benchmark generation time was on the order of minutes.

8.4 Availability of Source Code

The code for the first experiment with cairoVM can be found in <https://github.com/scaraven/stark-sphincs-aggr>.

The code for the second experiment with leanVM can be found in <https://github.com/scaraven/leanMultisig>.

Appendix A

Schnorr Signatures

Schnorr signatures are a classical elliptic-curve-based digital signature scheme invented by Claus Schnorr [63]. To perform Schnorr signature signing, we take a group G of prime order q , generator g and hash function $H : \{0, 1\}^* \rightarrow \mathbb{Z}/q\mathbb{Z}$ (often SHA-256).

A.1 Key Generation

Choose a private key $x \leftarrow \mathbb{Z}/q\mathbb{Z}$ and calculate the public key $y = g^{-x}$.

A.2 Signing

Given a message $M \leftarrow \{0, 1\}^*$

1. generate a random $k \leftarrow \mathbb{Z}/q\mathbb{Z}$
2. calculate $r = g^k$
3. calculate $e = H(r||M)$ where $||$ represents concatenation and we represent r as a bit string
4. calculate $s = k + xe$

The signature is returned as (s, e)

A.3 Signature Verification

1. Calculate $r_v = g^s y^e$
2. Let $e_v = H(r_v||M)$

Accept the signature if $e_v = e$

A.4 Schnorr Signature CairoVM Implementation

The two main costs from Schnorr signature verification are the SHA-256 invocation and elliptic curve group operations, which have non-trivial equations. The inclusion of Schnorr within cairoVM is an interesting comparison which highlights that despite Schnorr being an extremely simple conceptually, the combination of group operations and a singular SHA-256 invocation can make it relatively expensive to much more "computational" heavy signature schemes such as SPHINCS+.